

C++

Introduction

C++ is an object oriented programming language. It was developed by Bjarne Stroustrup at AT&T(American Telephones and Telegraphs) BELL labs in USA in early 1980s.

Stroustrup initially called the new language as " C with classes ". However later in 1983 the name was changed to C++. The idea of C++ comes from the C increment operator "++", there by suggesting that C++ is an incremented version of C. C++ is a superset of C language. Most of what we already know about C applies to C++ also. Therefore all most all C programs are also C++ programs.

Features of C++ :

- 1 C++ is a versatile programming language for handling very large programs.
- 2 It is suitable for virtually any programming task including development of editors, compilers, data bases, communication systems and complex real time applications.

3 C++ allows us to create hierarchy (tree structure) related objects.

4 C++ programs are easily maintainable and expendable.

5 It is expected that C++ will replaced C as general purpose language in the near future.

C++ Tokens :

The smallest individual elements or units in a program are called as Tokens. C++ has following tokens.

- 1 Identifiers
- 2 Keywords
- 3 Constants
- 4 Operators
- 5 Special characters

Identifiers :

Identifiers refer to the name of the variables, functions, arrays, classes ,etc created by the programmer using the combination of following characters.

- | | | | | |
|--------------|---|--------|-----|--------|
| 1 Alphabets | : | A to Z | and | a to z |
| 2 Digits | : | 0 to 9 | | |
| 3 Underscore | : | — | | |

Note :

- 1 The first character of an identifier must be an alphabet or underscore. we cannot use digit.
- 2 Default identifier length is 32 characters.

Keywords

Keywords are the words whose meaning has been already explained by the compiler. That means at the time of designing a language, some words reserved to do a specific tasks, such words are called key words or reserved words. All C compilers support 32 keywords, and C++ compilers supports 48 keywords whereas ANCII C++ has 15 extra latest keywords (total 63).

They are:

1. asm
2. auto
3. break
4. case
5. catch
6. char
7. class
8. const
9. continue
10. default
11. delete
12. do
13. double
14. else
15. enum
16. extern

17.float
18.for
19.friend
20.goto
21.if
22.inline
23.int
24.long
25.new
26.operator
27.private
28.protected
29.public
30.register
31.return
32.short
33.signed
34.sizeof
35.static
36.struct
37.switch
38.template
39.this
40.throw
41.try
42.typedef
43.union
44.unsigned
45.virtual

- 46. void
- 47. volatile
- 48. while

Added by ANSI C++ keywords

- 1. bool
- 2. const_cast
- 3. dynamic_cast
- 4 . explicit
- 5. export
- 6. false
- 7. mutable
- 8. namespace
- 9. reinterpret_cast
- 10. static_cast
- 11. true
- 12. typeid
- 13. typename
- 14. using
- 15. wchar_t

constants:

constants refer to fixed values, that don't change during the execution of program. C++ supports the following constants.

- 1 Integer constants
- 2 Character constants
- 3 Real or floating constants
- 4 String constants
- 5 Constant functions and objects

Operators:

It is a symbol which performs particular operation. C++ supports a rich set of operators. All C operators are valid in C++ also.

Special characters:

All characters other than alphabets and digits are treated as special characters.

C++ Data types:

cout:

It is a predefined object of standard output stream and is used to print data on standard output device (monitor).

Syntax:

```
cout << data;  
    or  
cout<<data_1<<data_2<<.....<<data_n;
```

Example:

```
cout<<"welcome to C++ programming";
```

The symbol " << " is called insertion operator, it sends the data on its right to the object on its left.

iostream.h

—

Program :

```
//Turbo c++ program
```

```
#include<iostream.h>
```

```
#include<conio.h>
void main()
{
    clrscr();
    cout<<"welcome to C++ programming";
    getch();
}
```

// ANSI c++ program

```
#include<iostream>
using namespace std;
int main()
{
    cout<<"welcome to C++ programming";
    return 0;
}
```

cin:

It is a predefined object of standard input stream and is used to read data from standard input device (key board).

Syntax:

```
cin >> variable;
    or
cin >> variable_1 >>variable_2>>.....>>variable_n;
```


Example:

```
int n;  
cin>>n;
```

The symbol “ >> ” is called extraction operator, it takes the data from standard input device and assigns it to the variable on its right.

Eg:

```
Int a,b;  
Cin>>a;  
Cin>>b;  
Cin>>a>>b;
```

Program : cin.cpp

```
#include<iostream.h>
```

```

#include<conio.h>
void main()
{
    int n;
    clrscr();
    cout<<"enter any number"
    cin>>n;
    cout<<"the value of n="<<n;
    getch();
}

```

Program:

```

#include<iostream.h>
#include<conio.h>
void main()
{
    int n=17;
    float f=18.14;
    double d=1814.1714;
    char c="R";
    clrscr();
    cout<<" number is : "<<n<<endl;
    cout<<" float is : "<<f<<endl;
    cout<<" double is : "<<d<<endl;
    cout<<" character is : "<<c<<endl;
    getch();
}

```

Program :

```

#include<iostream.h>
#include<conio.h>
void main()
{
    nt n;
    float f;
    double d;
    char c;
    clrscr();
}

```

```

    cout<<"enter any number : ";
    cin>>n;
    cout<<"enter any float : ";
    cin>>f;
    cout<<"enter any double : ";
    cin>>d;
    cout<<"enter any character : ";
    cin>>c;
    cout<<" number is : "<<n<<endl;
    cout<<" float is : "<<f<<endl;
    cout<<" double is : "<<d<<endl;
    cout<<" character is : "<<c<<endl;
    getch();
}

```

Flexible declaration :

C requires that all variables should be declared before the first executable statement. As against this, C++ allows declaration of variables at the point where they are used.

Program : *flex.cpp*

```

#include<iostream.h>
#include<conio.h>
void main()
{
    int n=10;
    cout<<" number is : "<<n<<endl;
    int m=20;
    cout<<" number is : "<<m<<endl;
    getch();
}

```

If we execute this program in C++, it will print the values of n and m successfully,

because in C++ we can declare variables at any point where they are used.

Operators in C++:

All C operators are valid in C++ also. In addition C++ introduces some new operators. They are

1. << insertion operator
2. >> extraction operator
3. :: scope resolution operator
4. endl end of line(similar to \n in C)
5. setw set width(similar to \t in C)
6. new and delete
7. . and ->
8. .* and ::*

Scope resolution operator (::)

It is the most important operator in C++. It plays several roles in C++ programming. It allows access to the global version of the variable. This is the first use of scope resolution operator.

Example : gloabal.cpp

```
#include<iostream.h>
#include<conio.h>
int n=10;
void main()
{
    int n=20;
    clrscr();
    cout<<"local n="<<n<<endl;
```

```

cout<<"global n="<<::n<<endl;
n=30;
::n=40;
cout<<"local n="<<n<<endl;
cout<<"global n="<<::n<<endl;
getch();
}

```

Output:

```

local n=20
global n=10
local n=30
global n=40

```

Note:

In C, if we declare global and local variable with the same name (i.e. n), it will prints the value of local variable n(i.e. n=20 but not n=10), where as in C++ we can declare global and local variable with the same name, and the global variables are accessed with the help of scope resolution operator.

Reference variables:

C++ introduces a new kind of variables known as reference variables. A reference variable provides an alias for a previously defined variables.

Syntax:

```
datatype  reference_variable =predefined_variable;
```

Eg:

```

int a;
int &b=a;

```

Example: ref_var.cpp

```

#include<iostream.h>
#include<conio.h>
void main()

```

```

{
    int a=100;
    int &b=a;
    cout<<"a"<<a<<endl;
    cout<<"b"<<b<<endl;
    b=200;
    cout<<"a"<<a<<endl;
    cout<<"b"<<b<<endl;
    getch();
}

```

Output:

```

A=100
B=100
A=200
B=200

```

enum:

It is a user defined data type and is used to define a set of constants of type int.

Syntax:

```
enum [ <type_tag> ] { <constant_name> [=value], ..... } [var_list];
```

Example: *enum.cpp*

```

#include<iostream.h>
#include<conio.h>
enum {a,b,c,d}
void main()
{
    cout<<"A"<<a<<endl;
    cout<<"B"<<b<<endl;
    cout<<"C"<<c<<endl;
    cout<<"D"<<d<<endl;
}

```

Output:

A=0
B=1
C=2
D=3

Example: enum1.cpp

```
#include<iostream.h>
#include<conio.h>
enum {a,b=10,c,d}
void main()
{
    cout<<"A="<<a<<endl;
    cout<<"B="<<b<<endl;
    cout<<"C="<<c<<endl;
    cout<<"D="<<d<<endl;
    getch();
}
```

Output:

A=0
B=10
C=11
D=12

Example: enum2.cpp

```
#include<iostream.h>
#include<conio.h>
enum boolean {false,true}b;
void main()
{
    b=true;
    cout<<"B="<<b<<endl;
    b=false;
    cout<<"B="<<b<<endl;
}
```

Output:

B=1

B=0

Example: enum3.cpp

```
#include<iostream.h>
#include<conio.h>
enum boolean {false,true};
boolean iseven(int x)
{
    if(x%2==0)
        return true;
    else
        return false;
}
void main()
{
    int n;
    boolean b;
    clrscr();
    cout<<"enter any number:";
    cin>>n;
    b=iseven(n);
    if(b==true)
        cout<<"given number is even";
    else
        cout<<"given number is odd";
    getch();
}
```

typedef:

It is a user defined data type and is used to assign the symbol name<identifier> to the data type definition<type definition>

Syntax:

```
typedef <type_definition> <identifier>;
```


Example: typedef.cpp

```
#include<iostream.h>
#include<conio.h>
typedef int i;
typedef unsigned long ul;
void main()
{
    i a=10;
    ul b=20;
    clrscr();
    cout<<"A="<<a<<endl;
    cout<<"B="<<b<<endl;
    getch();
}
```

Output:

```
A=10
B=20
```

Preprocessor statements or Preprocessor directives:

The processor is a program that process the source code before it passes through the compiler. The commands used to control the processor is known as preprocessor directives.

These directives are divided into 3 categories.

1. File inclusion directives
2. Macro substitution directives
3. Compiler control directives

File inclusion directives:

#include:

It is a pre processor file inclusion directive and is used to include header files. It provides instructions to the compiler to link the functions from the c/c++ library.

Syntax:

```
#include "Header file name"  
Or  
#include <Header file name>
```

When the file name is included within “ “, the search for the file is made first the current directory and then the standard directories. Otherwise the file name is included within < >, the search for the file name is only in the standard directories.

Macro substitution directives:

Macro is a process where an identifier in a program is replaced by a predefined string or value.

#define:

It is a preprocessor statement and is used to define macros.

1. simple macro :

Syntax:

```
#define <identifier> <predefined_string or value>
```

Eg: #define n 100

Example: s_macro1.cpp

```
#include <iostream.h>  
#include <conio.h>  
#define n 200  
# define m "RAJI"  
void main()  
{  
    cout<<"N="<<n<<endl;  
    cout<<"M="<<m<<endl;  
    getch();  
}
```

```
}
```

Output:

N=100

M=RAJI

Example: s_macro2.cpp

```
#include<iostream.h>
#include<conio.h>
#define print cout
#define read cin
#define cls clrscr
#define vm void main
#define gt getch
#define n 100
vm()
{
    int m;
    cls();
    print<<"enter any number";
    read>>m;
    print<<"n="<<n<<endl;
    print<<"m="<<m<<endl;
    gt();
}
```

2. complex macro(or)Macro with arguments:

The preprocessor permits to define more complex and more useful form of replacements.

Syntax:

#define identifier(arg_1,arg_2,.....arg_n) definition

Eg:

```
#define square(x)    x*x
```

Example: *c_macro.cpp*

```
#include<iostream.h>
#include<conio.h>
#define square(s) s*s
void main()
{
    int a,res;
    clrscr();
    cout<<"enter any number:";
    cin>>a;
    res=square(a);
    cout<<"square of given number="<<res;
    getch();
}
```

Example: *c_macro1.cpp*

```
#include<iostream.h>
#include<conio.h>
# define square(x) x*x
# define vm void main
#define pf cout
#define sf cin
#define cl clrscr
#define gt getch
vm()
{
    cl();
    int n,sqr;
    pf<<"Enter a number : ";
    sf>>n;
    sqr=square(n);
    pf<<"Square          = "<<sqr;
    gt();
}
```

```
printf("control string",[arg1,arg2....]);
```

Functions in C++:

Function :

It is a self contained block of statements and it performs a particular task. It can be used at several times in a program, but defined only once.

They are of 2 types.

1. Library functions
2. User defined functions.

Library functions:

The functions which are built with the compiler.

Eg:

- 1 printf
- 2 scanf
- 3 clrscr
- 4 etc.

Userdefined functions:

User can defined functions to do a task relavant to their programs, such functions are called as user defined functions.

Any function has 3 things.

1. Function declaration
2. Function definition
3. Function calling

In case of Library functions, the function declaration is in header files, function definition is in C or C++ libraries and function calling is in source

program. In case of user defined functions all three things are in source program.

Function declaration:

Syntax:

```
return_type    function_name ([arguments_list]);
```

Function definition:

Syntax:

```
return_type    function_name ([arguments_list])
{
    statements;
}
```

Function calling :

Syntax:

```
function_name ([parameter_list]);
```

Note:

The arguments which are given at the time of function declaration or function definition are called as arguments or formal arguments. The arguments which are given at the time of function calling are called as parameters or actual arguments.

Example: func.cpp

```
#include<iostream.h>
#include<conio.h>
void main()
{
    void disp();                declaration
    clrscr();
```

<pre> disp(); getch(); } </pre>	calling
<pre> void disp() { cout<<"Have a nice day"; } </pre>	definition

Rules for creating and accessing functions:

1. A function can be called any number of times.
2. A function may or may not receive arguments.
3. A function may or may not return a value
4. If A function doesnot return any value, the function return datatype will be specified as void.
5. If a function returns a value, only one value can be returned.
6. If a function returnrs a value the returning value must be returned with statement return.
7. If a function returns a value the execution of the return statement should be last.
8. If we cannot specify any return type,the function

returns an int by default.

9. A function returns a value, the returning value should be match with the function return data type.
- 10.If a functions reaturns a value , the returning value is replaced by a function calling statement.
- 11.A function is executed when the function is call by its name.
- 12.A function is defined after or before the main function.
- 13.Before calling a function, the function declaration or definition is must and should.
- 14.If a function definition is specified before the function call, then the function declaration is not necessary.
- 15.The function definition should not be terminated with a semicolon(;

return(keyword):

Exits immediately from the currently executing function to the calling routine, optionally returning a value.

Syntax:

return [<expression>];

Eg:

```
double square(double x)
{
    return x*x;
```



```
}
```

Example : func.cpp

```
#include<iostream.h>
#include<conio.h>
void main()
{
    int sum(int,int);
    int a,b,s;
    cout<<"enter 2 number";
    cin>>a>>b;
    s=sum(a,b);
    cout<<"sum="<<s;
    getch();
}
int sum(int x,int y)
{
    return x+y;
}
```

Function categories (or) Function prototypes:

A function, depending on whether arguments are present or not and whether a value is returning or not may belongs to one of the following categories.

1. Function with no arguments and return no value.
2. Function with arguments and no return value.
3. Function with arguments and return value.
4. Function with no arguments and return value.

Function with no arguments and no return value:

In this type the function has no arguments, it doesnot receive any data from the calling function. Similarly it doesnot return any value, the calling function

doesnot receive any data from called function. So there is no data communication between calling function and called function.

Example : *func1.cpp*

```
#include<iostream.h>
#include<conio.h>
void main()
{
    void sum();
    clrscr();
    sum();
    getch();
}
void sum()
{
    int a,b;
    cout<<"enter 2 numbers:";
    cin>>a>>b;
    cout<<"sum="<<a+b;
}
```

Function with arguments and no return value:

No

Yes

In this type the function has some arguments, it receives data from the calling function. But it doesnot return any value, the calling function doesnot receive any data from the called function. So there is one way data communication between

calling function and called function.

Example: func2.cpp

```
#include<iostream.h>
#include<conio.h>
void main()
{
    void sum(int,int);
    int a,b,s;
    clrscr();
    cout<<"enter 2 number";
    cin>>a>>b;
    sum(a,b);
    getch();
}
void sum(int x,int y)
{
    cout<<"sum="<< x+y;
}
```

Function with arguments and return value:

Yes

Yes

In this type the function has some arguments, it receives data from the calling function. Similarly it returns a value, the calling function receives data from the called function. So there is two way data communication between calling function

and called function.

Example : *func3.cpp*

```
#include<iostream.h>
#include<conio.h>
void main()
{
    int sum(int,int);
    int a,b,s;
    clrscr();
    cout<<"enter 2 number";
    cin>>a>>b;
    s=sum(a,b);
    cout<<"sum="<<s;
    getch();
}
int sum(int x,int y)
{
    return x+y;
}

}
```

Function with no arguments and return value:

Yes

No

In this type the function has no arguments, it doesnot receive any data from the calling function. But it returns a value, the calling function receives data from the called function. So there is one way data communication between calling function and called function.

Example : *func4.cpp*

```
#include<iostream.h>
#include<conio.h>
void main()
{
    int sum();
    int s;
    clrscr();
    s=sum();
    getch();
}
int sum()
{
    int x,y;
    cout<<"enter 2 numbers:";
    cin>>x>>y;
    return x+y;
}
```

Program : *nat_nos.cpp*

Program to display natural numbers from 1 to given number.

```
#include<iostream.h>
#include<conio.h>
#include<iomanip.h>
void main()
{
    void disp();
    int n;
    cout<<"enter any number:"
    cin>>n;
```

```

    cout<<"Natural numbers from 1 to"<<n<<":"<<endl;
    disp(n);
    getch();
}
void disp(int x)
{
    int i;
    for(i=1;i<=x;i++)
    {
        cout<<setw<<I;
    }
}

```

Program : max1.cpp

Program to display maximum number of given 2 numbers.

```

#include<iostream.h>
#include<conio.h>
void main()
{
    int max(int,int);
    int a,b,m;
    clrscr();
    cout<<"enter 2 numbers";
    cin>>a>>b;
    m=max(a,b);
    cout<<"Maximum of given 2 numbers is :"<<m;
    getch();
}

int max(int x,int y)
{
    if(x>y)
        return x;
    else
        return y;
}

```

Program : max2.cpp

Program to display maximum number of given 2 numbers by using ternary operator.

```
#include<iostream.h>
#include<conio.h>
void main()
{
    int max(int,int);
    int a,b,m;
    clrscr();
    cout<<"enter 2 numbers";
    cin>>a>>b;
    m=max(a,b);
    cout<<"Maximum of given 2 numbers is :"<<m;
    getch();
}

int max(int x,int y)
{
    int m;
    m=x>y?x:y;
    return m;
}
```

Program : max3.cpp

Program to display maximum number of given 3 numbers.

```
#include<iostream.h>
#include<conio.h>
void main()
{
    int max(int,int);
    int a,b,c,m;
    clrscr();
    cout<<"enter 3 numbers";
    cin>>a>>b>>c;
    m=max(a,b);
    m=max(m,c
```

```

        cout<<"Maximum of given 3 numbers is :"<<m;
        getch();
    }

int max(int x,int y)
{
    if(x>y)
        return x;
    else
        return y;
}

```

Program : max4.cpp

Program to display maximum number of given 2 number by using ternary operator.

```

#include<iostream.h>
#include<conio.h>
void main()
{
    int max(int,int);
    int a,b,c,m;
    clrscr();
    cout<<"enter 3 numbers";
    cin>>a>>b>>c;
    m=max(a,b);
    cout<<"Maximum of given 2 numbers is :"<<m;
    getch();
}

int max(int x,int y)
{
    int m;
    m=x>y?x:y;
    return m;
}

```

Program : fn_fact.cpp

To find factorial of given number.

```
#include<iostream.h>
#include<conio.h>
void main()
{
    unsigned long fact(int);
    unsigned long f;
    int n;
    clrscr();
    cout<<"Enter a number : ";
    cin>>n;
    f=fact(n);
    cout<<"Factorial of given number : "<<f;
    getch();
}
unsigned long fact(int x)
{
    unsigned long f=1;
    while(x>=1)
    {
        f=f*x;
        x--;
    }
    return f;
}
```

Program : *fn_rev.cpp*

To find reverse number of given number

```
#include<iostream.h>
#include<conio.h>
void main()
{
    unsigned long rev(int);
    unsigned long r;
    int n;
    clrscr();
    cout<<"Enter a number : ";
```

```

    cin>>n;
    r=rev(n);
    cout<<"Revrse number  : "<<r;
    getch();
}
unsigned long rev(int x)
{
    unsigned long rv=0;
    while(x>0)
    {
        rv=(rv*10)+(x%10);
        x=x/10;
    }
    return rv;
}

```

Recursive Functions:

Calling a function with the same function definition is known as recursive function.

If we want to work with recursive functions, we must follow the following 2 aspects.

- 1 Calling by itself.
- 2 Termination condition.

Program : *rfn_nnos.cpp*

Program to display natural numbers from 1 to given number using recursive function.

```

#include<iostream.h>
#include<conio.h>
#include<iomanip.h>
void main()
{
    void disp(int x);
    int n;
    clrscr();
    cout<<"enter any number:";
    cin>>n;
}

```

```

        cout<<"Natural numbers from 1 to given number "<<n<<":.";
        disp(n);
        getch();
    }
    void disp(int x)
    {
        if(x>1)
            disp(x-1);
        cout<<setw(5)<<x;
    }

```

Program : rfn_fact.cpp

Program to find factorial of given number using recursive function.

```

#include<iostream.h>
#include<conio.h>
void main()
{
    unsigned long fact(int n);
    int n;
    unsigned long f;
    clrscr();
    cout<<"enter any number:";
    cin>>n;
    f=fact(n);
    cout<<"Factorial of "<<n<<":."<<f;
    getch();
}
unsigned long fact(int x)
{
    if(x==1)
        return 1;
    else
        return x*fact(x-1);
}

```

Program : rfn_rev.cpp

To find reverse number of given number using recursive function

```
#include<iostream.h>
#include<conio.h>
void main()
{
    unsigned long rev(int);
    unsigned long r;
    int n;
    clrscr();
    cout<<"Enter a number : ";
    cin>>n;
    r=rev(n);
    cout<<"Reverse number  : "<<r;
    getch();
}
unsigned long rev(int x)
{
    static unsigned long rv=0;
    rv=(rv*10)+(x%10);
    x=x/10;
    if(x>0)
        rev(x);
    return rv;
}
```

Functions with default arguments:

C++ allows us to call a function without specifying all its arguments. In such cases the function assigns a default value to the parameter, which doesnot have a matching arguments in the function call. Default values are specified when the function is declared or defined.

Example: def_arg.cpp

```

#include<iostream.h>
#include<conio.h>
void fun(int x=10,int y=20)
{
    cout<<"X="<<x<<endl;
    cout<<"Y="<<y<<endl;
}
void main()
{
    clrscr();
    cout<<"calling-1"<<endl;
    fun();
    cout<<"calling-2"<<endl;
    fun(30);
    cout<<"calling-3"<<endl;
    fun(40,50);
    getch();
}

```

we must assign values to the

Output:-

```

Calling-1
X=10
Y=20
Calling-2
X=30
Y=20
Calling-3
X=40
Y=50

```

Function overloading

It is a concept used to define more than one function with the same name, But it differs atleast in any one of the following aspects.

- 1 Number of arguments may be differ.
- 2 Type of arguments may be differ.
- 3 Order of arguments may be differ.

Example: fn_poly.cpp

```
#include<iostream.h>
#include<conio.h>
void line(int r)
{
    for(int i=20;i<=60;i++)
    {
        gotoxy(i,r);
        cout<<(char)196;
    }
}
void line(int r,int sc,int ec)
{
    for(int i=sc;i<=ec;i++)
    {
        gotoxy(i,r);
        cout<<(char)196;
    }
}
void line(int r,int sc,int ec,char ch)
{
    for(int i=sc;i<=ec;i++)
    {
        gotoxy(i,r);
        cout<<ch;
    }
}
void main()
{
    clrscr();
    line(5);
    line(10,20,60);
    line(15,20,60,196);
    line(20,40,60,'*');
```

```
    getch();  
}
```

inline function :

An inline function is a function, that is expanded inline when it is invoked. i.e. the compiler replaces the function call with the corresponding function code. (something similar to macro expansion).

Syntax :

```
inline returntype function_name([arg_list])  
{  
    Statements;  
  
}
```

Example:

```
#include<iostream.h>  
#include<conio.h>  
inline void func()  
{  
    cout<<"welcome to CPP programming";  
}  
void main()  
{  
    clrscr();  
    func();  
    getch();  
}
```

Note : Only small definition of functions can be declared as inline. i.e. if we use loops in inline function the compiler will display error message.

ellipsis(...):

An ellipsis(...) consists of 3 successive periods with no white spaces intervening.

We can use an ellipsis in the formal argument lists of function prototypes to indicate a variable number of arguments or arguments with varying types.

Example:

```
void func(...)  
{  
  
    cout<<"Good Morning";  
}  
void main()  
{  
    clrscr();  
    func();  
    func(10);  
    func(1,2,3,4,5);  
    func('a','b','c');  
    func("raji");  
    getch();  
}
```

Output

```
Good Morning  
Good Morning  
Good Morning  
Good Morning  
Good Morning
```


va_list:

It is in the header file <stdarg.h>. It is an array that holds information needed by va_arg and va_end.

When a called function takes a variable argument list, it declares a variable of type va_list.

va_arg, va_end, va_start :

These are macros that implements variable arg list.

Syntax :

```
void va_start(va_list vl, last fix);  
type va_arg(va_list, type);  
void va_end(va_list vl);
```

Example: ellipsis.cpp

```
#include<iostream.h>  
#include<conio.h>  
#include<stdarg.h>  
int sum(int x,int y,...)  
{  
    int s,n;  
    va_list vl;  
    s=x+y;  
    va_start(vl,y);  
    n=va_arg(vl,int);
```

```

while(n!=0)
{
    s=s+n;
    n=va_arg(vl,int);
}
va_end(vl);
return s;
}
void main()
{
    clrscr();
    cout<<sum(10,20);
    cout<<endl;
    cout<<sum(10,20,30);
    cout<<endl;
    cout<<sum(10,20,30,40);
    cout<<endl;
    cout<<sum(1,2,3,4,5,6,7,8,9,10);
    cout<<endl;
    getch();
}

```

Introduction to OOP'S :

It is an object oriented programming introduced in 1980's by Bjarne Stroustrup. OOP'S improves the style of a program. It allows the user to develop a program structure, which is representing a physical object. This concept mainly introduces a mechanism called ***class***.

Class : It is a user defined data type and is used to define data variables and methods(functions).

Object : It is an instance of a class. In otherwords it

represents one's specific organizational elements.

Data members : Since every organizational element(object) consists certain information that can be represented with the help of data members.

Member functions or methods : These are the functions used to perform the special operations related to the organizational elements(objects).

Features of OOPS :

- 1 . Emphasis is on data rather than procedures.
- 2 . Programs are divided into what are known as objects.
- 3 . Data structures are designed such that they characterize the objects.
- 4 . Data is hidden and can't be accessed by external functions.
- 5 . Objects may communicate with each other through functions.
- 6 . New data and functions can be easily added whenever necessary.

Basic characteristics (or) Basic concepts of OOPS

- 1 Objects
- 2 Classes
- 3 Data encapsulation and Data abstraction

4 Inheritance

5 Polymorphism

Objects :

Objects are the runtime entities in object oriented programming. They may represents a person, a place, etc. that are program has to handle. They may also represent user defined data.

Classes :

The objects contains data and code to manipulate that data, the entire set of data and code of an object can be made a user defined data type such as class. Once a class has been defined, we can create any number of objects belongs to that class.

Data encapsulation and Data abstraction :

Data encapsulation : (Data hiding)

It is a mechanism that associates the code and data manipulations into a single unit and keeps them safe from external interface. This is supported by the construct called ***class***.

The use of encapsulation is to protecting the members of a class. The data is not accessible to the outside class and only those functions which are wrapped (i.e. within the class) in the class can access it. This

insulation of data from direct access by the program is called as ***Data hiding.***

Data abstraction:

abstract data type: a set of operations performed in a particular data type is known as ***abstract data type.***

The technique of creating new ***abstract*** data types that are well suited to an application is known as ***data abstraction.***

Data abstraction means we can combine the data structures and operations on the data structures together into a new abstract data type. An abstract data type behaves just like a data types that are not a part of language definition, they are created by the user.

Inheritance :

It is a mechanism which is used to extend the definition of existing class. The extension can be done by declaring some other class. The class that is originally present is called as Base class and the class which is extended with the help of inheritance is called as Derived class. A derived class always can access the members of base class, whereas a base class never access the members of derived class. The main purpose of inheritance is reusability of definition that is already made.

Polymorphism :

The word polymorphism is derived from 2 latin words

- 1 Poly (many)
- 2 Morphs (forms)

Polymorphism is nothing but the ability to access different implementations of the function using the same name.

There are 2 levels at which polymorphism operates.

- 1 Compile time polymorphism (function and operator overloading)
- 2 Run time polymorphism (virtual functions)

Basic structure of C++ program :

[Document section]
Preprocessor section (or) Link section
class definitions
main Function section

Document section :

It is an optional section. This section consists a set of comment lines giving the name of program, author name and some other details of the program.

pre-processor (or) Link section :

It provides instructions to the compiler to link the functions

from the c/c++ library.

Class definitions:

Any C++ Program must contains class definitions. Classes are primary and essential elements of C++ program. This section consists a set of class definitions and its data manipulations.

Main function section :

Every C++ program consists one main function. The main function creates objects of classes (pre or user defined) and establishes communication between them.

Classes and Objects

class

It is a user defined data type and is used to define data members and member functions.

General form of class declaration :

```
class classname
{
    private :
        declaration of variables;
        declaration / definition of functions

    public:
        declaration of variables;
        declaration /definition of functions

};
```

The class declaration is similar to ***struct*** declaration. The keyword ***class*** is used to declare a class. The body of a class is enclosed with curly braces and terminated by a semi colon. The class body contains declaration of variables, and declaration or definition of functions. The keywords private and public are access specifiers.

private :

Every variable and member function declared between private keyword and then the access specifier (or) end of the class is hidden from the users of the class. They are however accessible to the member functions of the class itself.

public :

Every variable and member functions declared between public keyword and the end of the class is available to the users(out side class) as well as member functions of the class itself.

Note:

If there is no access specifier keyword appears before the declaration of any member in a class, private is assumed.

Object :

It is an instance of a class, in otherwords it represents one specific organizational element.

Declaration :

Object declaration is similar to variable declaration. The necessary memory space is allocating to an objects at this stage.

Syntax:

```
classname  objname ;  
           or  
classname  objname-1,objname-2,.....,objname-n ;
```

Eg:

```
        emp e;  
        (or)  
        emp e1,e2,e3;
```

Accessing class members:

1. Using dot (.)operator :

It is used to access the members of a class with the help of normal object.

object_name . class_member ;

2. Using arrow (->) operator :

It is used to access the members of a class with the help of object pointer.

object_pointer -> class_member;

Example :++

```
#include<iostream.h>
#include<conio.h>
class test
{
    public:
        int n;
};
void main()
{
    test t;
    clrscr();
    cout<<"enter a number into object :"<<endl;
    cin>>t.n;
    cout<<"data in object :"<<t.n;
}
```

Defining member functions inside the class

It means a member function is to replace the function declaration by the actual function definition inside the class.

When a function is defined inside a class, it is treated

as an inline function. Therefore all the restrictions and limitations that apply to an inline function, are also applicable here. Normally only small functions are defined inside the class.

Example :

```
#include<iostream.h>
#include<conio.h>
class test
{
    private:
        int n;
        object t

    public:
        void accept()
        {
            cin>>n;
        }
        void print()
        {
            cout<<n;
        }
};
void main ()
{
    test t;
    clrscr();
    cout<<"enter any data into object";
    t.accept();
    cout<<"given data is :";
    t.print();
    getch();
}
```

data method area

Defining member functions outside the class :

Member functions that are declare inside a class have to be defined separately outside the class. These definitions are very much similar to normal C++ functions.

Syntax:

```
returntype  classname :: fuctionname( [argumentlist] )
{
    statements;
}
```

Example:

```
#include<iostream.h>
#include<conio.h>
class test
{
    private:
        int n;
    public:
        void accept();
        void display();
};
void test::accept()
{
    cin>>n;
}
void test::display()
{
    cout<<n;
}
void main()
{
    test t;
    clrscr();
```

```

    cout<<"enter a number:";
    t.accept();
    cout<<"given number is:";
    t.display();
    getch();
}

```

Accepting strings:

Example:

```

#include<iostream.h>
#include<conio.h>
void main()
{
    char c[20];
    clrscr();
    cout<<"enter a string";
    cin>>c;
    cout<<c;
    getch();
}

```

Output

If we give input	WELCOME
The output will be	WELCOME
And if we give input	WELCOME TO C++
The output will be	WELCOME

cin object can't read spaces in a string variable.

If we want to read a string with spaces into a string variable, we use getline method.

getline : (istream::getline member function)

It is an input stream function and is used to read a string

with spaces from standard input stream.

Syntax :

```
istream& getline(char *s ,int n , char ch='\n');
```

Example:

```
#include<iostream.h>
#include<conio.h>
void main()
{
    char c[20];
    clrscr();
    cout<<"enter a string";
    // cin>>c; //
    cin.getline(c,20)
    cout<<c;
    getch();
}
```

Output

If we give input
The output will be

WELCOME TO BDPS
WELCOME TO BDPS

Skipping problem :

Example:

```
#include<iostream.h>
#include<conio.h>
void main()
{
    char c[20];
```

```

int n;
clrscr();
cout<<"enter any integer :"<<endl;
cin>>n;
cout<<"enter a string"<<endl;
cin.getline(c,20)
cout<<"given string :"<<c<<endl;
cout<<"given number:"<<n<<endl;
getch();
}

```

Output

```

enter any integer:100
enter a string :
given string:
given number:100

```

enter any integer :

n

c

new line character, hence skipped

In the above program, it can't read string data into string variable because the integer reading statement creates a new line character in input stream. That will assign to the string variable. If we want to avoid this problem use the following method.

ignore : (istream::ignore member function)

It is a input stream function. It ignores specified number of characters from standard input stream.

Syntax:

istream& ignore(int n=1);

Example:

```
#include<iostream.h>
#include<conio.h>
void main()
{
    char c[20];
    int n;
    clrscr();
    cout<<"enter any integer :"<<endl;
    cin>>n;
    cout<<"enter a string"<<endl;
    cin.ignore(); // cin.ignore(1)
    cin.getline(c,20);
    cout<<"given string :"<<c<<endl;
    cout<<"given number:"<<n<<endl;
    getch();
}
```

setprecision :

It is a manipulator and it changes precision to 'n' places after the decimal point for floating values.

Syntax : setprecision(int n);

setiosflags:

It is a manipulator and is used to set specified iosflag.

Syntax: `setiosflags(flag);`

flags:

1. `ios::showpoint` → it displays decimal point for floating values
2. `ios::fixed` → fixed floating point notation
3. `ios::showpos` → it displays + sign before the +ve values

resetiosflags :

syntax:

`resetiosflags(flag)`

Example :

```
#include<iostream.h>
#include<conio.h>
#include<iomanip.h>
void main()
{
    float f=10.4389;
    cout<<"F="<<f;
    cout<<setprecision(2);
    cout<<"F="<<f;
}
```

Output

F=10.4389

F=10.44

Example :

```
#include<iostream.h>
#include<conio.h>
```

```
#include<iomanip.h>
void main()
{
    float f=1024;
    cout<<setprecision(2);
    cout<<setiosflags(ios::showpoint);
    cout<<setiosflags(ios::fixed);
    cout<<setiosflags(ios::showpos);
    cout<<"F="<<f;
}
```

Output

F=+1024

resetiosflags : It removes specified iosflags.

Syntax : resetiosflags(flag);

Example :

```
#include<iostream.h>
#include<conio.h>
#include<iomanip.h>
void main()
{
    float f=10;
    cout<<"F="<<f;
    cout<<setiosflags(ios::showpoint);
    cout<<"F="<<f;
    cout<<setiosflags(ios::showpos);
    cout<<setprecision(2);
    cout<<"F="<<f;
    cout<<resetiosflags(ios::showpos);
    f=5000;
    cout<<setiosflags(ios::fixed);
    cout<<"F="<<f;
    getch();
}
```

Output

F=10
F=10.000000
F=+10.00
F=5.00e+03
F=5000.00

Program : *edetails.cpp*

Write a program to enter employ number, employ name, and employ salary and to display them using class.

```
#include<iostream.h>
#include<conio.h>
#include<iomanip.h>
class employee
{
    private:
        int eno;
        char ename[25];
        float esal;
    public:
        void accept();
        void display();
};
void employee::accept()
{
    cout<<"enter employee number:";
    cin>>eno;
    cout<<"enter employee name:";
    cin.ignore();
    cin.getline(ename,25);
    cout<<"enter employee salary:";
    cin>>esal;
}
void employee::display()
{
    cout<<"employee number:"<<eno<<endl;
    cout<<"employee name:"<<ename<<endl;
    cout<<setprecision(2);
```

```

    cout<<setiosflags(ios::showpoint);
    cout<<setiosflags(ios::fixed); // if we want to add more than one records
                                     declare these 3 methods in
main() method.
    cout<<"employee salary:"<<esal;
}
void main()
{
    clrscr();
    employee e;
    cout<<"Enter Employee Details"<<endl<<endl;
    e.accept();
    clrscr();
    cout<<"Employee Details"<<endl<<endl;
    e.display();
    getch();
}

```

Program : sdetails.cpp

Write a program to enter student number, student name, marks in C, CPP, Java, and to calculate and display total marks, average, result and division.

```

#include<iostream.h>
#include<conio.h>
#include<iomanip.h>
#include<string.h>
class student
{
    private:
        int sno,c,cpp,java,tot;
        char sname[25],res[10],div[10];
        float avg;
    public:
        void accept();
        void calc();
        void display();
};
void student::accept()

```

```

{
    cout<<"enter student number  : ";
    cin>>sno;
    cout<<"enter student name    : ";
    cin.ignore();
    cin.getline(sname,25);
    cout<<"enter marks in C      : ";
    cin>>c;
    cout<<"enter marks in C++    : ";
    cin>>cpp;
    cout<<"enter marks in Java   : ";
    cin>>java;
}
void student::calc()
{
    tot=c+cpp+java;
    avg=(float)tot/3;
    if(c>=50 && cpp>=50 && java>=50)
    {
        strcpy(res,"PASS");
        if(avg>=60)
        {
            strcpy(div,"FIRST");
        }
        else
        {
            strcpy(div,"SECOND");
        }
    }
    else
    {
        strcpy(res,"FAIL");
        strcpy(div,"NO DIVISION");
    }
}
void student::display()
{
    cout<<"student number  : "<<sno<<endl;
    cout<<"student name    : "<<sname<<endl;
    cout<<"Marks in c      : "<<c<<endl;

```

```

cout<<"Marks in cpp    : "<<cpp<<endl;
cout<<"Marks in jaa    : "<<java<<endl;
cout<<"Total marks     : "<<tot<<endl;
cout<<"Average marks    : "<<avg<<endl;
cout<<"Result           : "<<res<<endl;
cout<<"Division         : "<<div<<endl;
}
void main()
{
    char ch;
    clrscr();
    student s;
    cout<<setprecision(2);
    cout<<setiosflags(ios::showpoint);
    cout<<setiosflags(ios::fixed);
    /* cout<<"Enter Student Details"<<endl<<endl;
    s.accept();
    s.calc();
    clrscr();
    cout<<"Student Details"<<endl<<endl;
    s.display(); */

    // if we want to add more then one record use do-while loop //

    do
    {
        clrscr();
        s.accept();
        s.calc();
        clrscr();
        s.display();
        cout<<"Do you want to continue (y/n) ";
        cin>>ch;
    }
    while(ch=='y');
    getch();
}

```

Local classes :

Classes that can be defined and used inside a function block, such classes are called as local classes.

Note :

Local classes doesnot support non inline functions. i.e. write the function definitions inside the class.

Program : *localcls.cpp*

```
#include<iostream.h>
#include<conio.h>
void func()
{
    class test
    {
        private:
            int n;

        public:
            void accept()
            {
                cin>>n;
            }
            void disp()
            {
                cout<<n<<endl;
            }
    };
    test t;
    cout<<"enter any number:";
    t.accept();
    cout<<"given number is:";
    t.disp();
}
```

```

}
void main()
{
    clrscr();
    func();
    getch();
}

```

Inner classes (or) class within a class (or) nested classes:

Defining a class within another class is known as inner class.

Note :

Inner classes doesnot support non inline functions. i.e. write the function definitions inside the class.

Accessing inner class inside the outer class :

Program : *nested1.cpp*

```

#include<iostream.h>
#include<conio.h>
class outer
{
    private:
    class inner
    {
        private:
        int n;
        public:
        void accept()
        {

```



```

        cin>>n;
    }
    void disp()
    {
        cout<<n;
    }
};
public:
    void inn_acc()
    {
        inner obj;
        cout<<"enter data in object:";
        obj.accept();
        cout<<"data in object:";
        obj.disp();
    }
};
void main()
{
    outer obj;
    clrscr();
    obj.inn_acc();
}

```

Accessing inner class outside the outer class :

Syntax :

outerclass_name :: innerclass_name object_name;

Program : nested2.cpp

```
#include<iostream.h>
```

```

#include<conio.h>
class outer
{
    private:
        class inner
        {
            private:
                int n;
            public:
                void accept()
                {
                    cin>>n;
                }
                void disp()
                {
                    cout<<n;
                }
        };
};

void main()
{
    outer::inner obj;
    clrscr();
    cout<<"enter data in object:";
    obj.accept();
    cout<<"given data in object:";
    obj.disp();
    getch();
}

```

Program : *nested3.cpp*

```

#include<iostream.h>
#include<conio.h>
class outer
{
    private:
        int x;
    class inner
    {
        private:
            int y;
        public:
            void accept()
            {
                cin>>y;
            }
            void disp()
            {
                cout<<y<<endl;
            }
    };
    public:
        void accept()
        {
            cin>>x;
        }
        void disp()
        {
            cout<<x<<endl;
        }
    };
};
void main()
{
    outer obj1;
    outer::inner obj2;
    clrscr();
    cout<<"Enter data in object 1 : ";

```

```

obj1.accept();
cout<<"Enter data in object 2 : ";
obj2.accept();
cout<<"Data in object 1 : ";
obj1.disp();
cout<<"Data in object 2 : ";
obj2.disp();
getch();
}

```

Static variables:

The memory of static variables remains unchanged until the end of the program. That means, it cannot reinitialize between the function calls.

Program : static.cpp

```

#include<iostream.h>
#include<conio.h>
#include<iomanip.h>
void func()
{
    int i=1;
    cout<<i<<setw(5);
    i++;
}

void main()
{
    clrscr();
    for(int i=1;i<=10;i++)
    {
        func();
    }
    getch();
}

```

Output

```

1      1      1      1      1      1      1      1      1      1

```

Here int i=1 is declared as **auto** (default). After incrementing i value becomes 2 and when the control will go to func(), again it is initialized to 1. Hence we will get 1 ten times, in the output.

Whereas if we declare i as static i.e. static int i=1, the value of i will not change during the execution of the program. The compiler will not execute the statement static int i=1, and executes other statements and hence i value will be incremented and we will get the output from 1 to 10.

Example :

```
#include<iostream.h>
#include<conio.h>
#include<iomanip.h>
void func()
{
    static int i=1;
    cout<<i<<setw(5);
    i++;
}

void main()
{
    clrscr();
    for(int i=1;i<=10;i++)
    {
        func();
    }
    getch();
}
```

Output

1 2 3 4 5 6 7 8 9 10

Static data members :

A data member of a class can be declared as static, the

properties of a static member variables are similar to a C static variable. It is initialized to zero, when the first object of its class is created.

When a data member is declared as static, it can be accessed with the help of any one of the object of that class. In other words, a static data member is commonly shared all the objects of the same class.

Note:

The type and scope of each static member variable must be defined outside the class definition. This is necessary because static data members are stored separately rather than as a part of an object

Example : *nonstat.cpp*

```
#include<iostream.h>
#include<conio.h>
class num
{
    private:
        int n;
    public:
        void accept()
        {
            cin>>n;
        }
        void disp()
        {
            cout<<n<<endl;
        }
};
void main()
```

object 1

object 2

```

{
    num n1,n2;
    clrscr();
    cout<<"Initial data in object 1:";
    n1.disp();
    cout<<"Initial data in object 2:";
    n2.disp();
    cout<<"enter data in object 1:";
    n1.accept();
    cout<<"Data in object 1:";
    n1.disp();
    cout<<"Data in object 2:";
    n2.disp();
    cout<<"enter data in object 2:";
    n2.accept();
    cout<<"Data in object 1:";
    n1.disp();
    cout<<"Data in object 2:";
    n2.disp();
    getch();
}

```

Output

Initial data in object 1: 7890 (Garbage Value)

Initial data in object 2: 0 (Garbage Value)

Enter data in object 1: 45

Data in object 1: 45

Data in object 2: 0

Enter data in object 2: 78

Data in object 1: 45

Data in object 1: 78

Initially garbage values are stored in the variables, if we change value of one variable, that value only changed and the values of other variables will have same garbage values.

Whereas in static variables, the initial values of the variables are zero and if the value of one variable changed, the other also will be changed.

Example : *staticfn.cpp*

```

#include<iostream.h>
#include<conio.h>
class num
{
object 1          object 2
private:
    static int n;
public:
    void accept()
    {
        cin>>n;
    }
    void disp()
    {
        cout<<n<<endl;
    }
};
int num::n;
void main()
{
    num n1,n2;
    clrscr();
    cout<<"Initial data in object 1:";
    n1.disp();
    cout<<"Initial data in object 2:";
    n2.disp();
    cout<<"enter data in object 1:";
    n1.accept();
    cout<<"Data in object 1:";
    n1.disp();
    cout<<"Data in object 2:";
    n2.disp();
    cout<<"enter data in object 2:";
    n2.accept();
    cout<<"Data in object 1:";
    n1.disp();
    cout<<"Data in object 2:";
    n2.disp();
    getch();
}

```


Output

```
Initial data in object 1: 0
Initial data in object 2: 0
Enter data in object 1: 45
Data in object 1: 45
Data in object 2: 45
Enter data in object 2: 78
Data in object 1: 78
Data in object 1: 78
```

Static member function :

When a member function is declared of type static, then it is possible to invoke the member functions directly, with the help of class name without declaring any objects of that class. A static function can have to access only other static members declare in the same class. A static member function can be called using classname as follows.

Syntax:

```
class_name                                ::
function_name([argument_list]);
```

Example : *statfn1.cpp*

```
#include<iostream.h>
#include<conio.h>
class num
{
```

```

private:
    static int n;
    static char c;
public:
    static void accept()
    {
        cout<<"enter any number:";
        cin>>n;
        cout<<"enter any character:";
        cin>>c;
    }
    static void disp()
    {
        cout<<"given number:";
        cout<<n<<endl;
        cout<<"given character:";
        cout<<c;
    }
};
int num::n;
char num::c;
void main()
{
    clrscr();
    num::accept();
    num::disp();
    getch();
}

```

Array of Objects :

We know that an array can be of any data type including struct. Similarly we can also have array of variables that are of the type class. Such variables are called array of objects.

Syntax :

```
class_name    object_name [size];
```

Eg : student s[100];

Program : arr_emp.cpp

To enter employ number, employ name and employ salary of 20 employs and display using array of objects.

```
#include<iostream.h>
#include<conio.h>
#include<iomanip.h>
class employ
{
    private:
        int eno;
        char ename[20];
        float esal;
    public:
        void accept();
        void disp();
};
void employ::accept()
{
    cout<<"Enter Employ number:";
    cin>>eno;
    cout<<"Enter Employ Name:";
    cin.ignore();
    cin.getline(ename,20);
    cout<<"Enter employ salary:";
    cin>>esal;
}
void employ::disp()
{
    cout<<eno<<setw(10)<<ename<<setw(10)<<esal<<endl<<endl;
}
void main()
{
    employ e[20];
    clrscr();
    int i,n;
    cout<<setprecision(2);
```

```

cout<<setiosflags(ios::showpoint);
cout<<setiosflags(ios::fixed);
cout<<"Enter number of records to be added:";
cin>>n;
for(i=0;i<n;i++)
{
    cout<<endl<<endl<<endl;
    cout<<"Enter Record No:"<<i+1<<endl<<endl<<endl;
    e[i].accept();
}
clrscr();
cout<<"EMP NO"<<setw(10)<<"EMP NAME"<<setw(10)<<
    "EMP SAL"<<endl<<endl;
cout<<"-----"<<endl;
for(i=0;i<n;i++)
{
    e[i].disp();
}
getch();
}

```

Program : arr_stu.cpp

To enter student number, student name, marks in C, CPP, Java, and to calculate and display total of 3 subjects, average, result and division of 100 students by using array of objects.

```

#include<iostream.h>
#include<conio.h>
#include<iomanip.h>
#include<string.h>
class student
{
    private:
        int sno,c,cpp,java,tot;
        char sname[20],res[10],div[10];
        float avg;
    public:
        void accept();
        void calc();
}

```

```

        void disp();
};
void student::accept()
{
    cout<<"Enter student number:";
    cin>>sno;
    cout<<"Enter student name:";
    cin.ignore();
    cin.getline(sname,20);
    cout<<"Enter marks in C,CPP,Java:";
    cin>>c>>cpp>>java;
}
void student::calc()
{
    tot=c+cpp+java;
    avg=(float)tot/3;
    if(c>=50 && cpp>=50 && java>=50)
    {
        strcpy(res,"PASS");
        if(avg>=60)
        {
            strcpy(div,"FIRST");
        }
        else
        {
            strcpy(div,"SECOND");
        }
    }
    else
    {
        strcpy(res,"FAIL");
        strcpy(div,"NO DIVISION");
    }
}
void student::disp()
{
    cout<<"Sno = "<<sno<<endl;
    cout<<"Sname = "<<sname<<endl;
    cout<<"Marks in C ="<<c<<endl;
    cout<<"Marks in CPP = "<<cpp<<endl;
}

```

```

        cout<<"Marks in Java = "<<java<<endl;
        cout<<"Total Marks = "<<tot<<endl;
        cout<<"Average = "<<avg<<endl;
        cout<<"Result = "<<res<<endl;
        cout<<"Division = "<<div<<endl<<endl;
    }
    void main()
    {
        student s[100];
        clrscr();
        int i,id=0;
        char ch;
        cout<<setprecision(2);
        cout<<setiosflags(ios::showpoint);
        cout<<setiosflags(ios::fixed);
        do
        {
            clrscr();
            cout<<"Enter Student Record : "<<id+1<<endl<<endl<<endl;
            s[id].accept();
            s[id].calc();
            id++;
            cout<<endl<<endl<<"Do you want to continue(Y/N) :";
            cin>>ch;
            cout<<endl<<endl;
        }
        while(ch!='n' && id<100); // (ch=='y' && id<100)
        clrscr();
        cout<<"  for(i=0;i<n;i++)
        {
            s[i].disp();
            cout<<"Press any key to continue..."<<endl;
        }
        getch();
    }
}

```

Passing object as function argument (or) parameter :

Like any other data type an object may be used as function argument or

parameter.

Returning object :

A function can not only receive objects as function arguments, but also can return them.

Example: obj_sum.cpp

```
#include<iostream.h>
#include<conio.h>
class test
{
    private:
        int n;
    public:
        void accept()
        {
            cin>>n;
        }
        void disp()
        {
            cout<<n;
        }
        test addition(test);
};
test test::addition(test obj)
{
    test result;
    result.n=n+obj.n;
    return result;
}
void main()
{
    clrscr();
    test t1,t2,t3;
    cout<<"enter any data in t1 object :";
```

implicit argument

explicit argument

```

t1.accept();
cout<<"enter any data in t2 object :";
t2.accept();
cout<<"Addition of 2 objects:";
t3=t1.addition(t2); // or t2.addition(t1)
t3.disp();
getch();
}

```

[In the statement **test test :: addition(test obj)** method, first test is return type of object, as we are receiving object as return value hence the return type is a class name, and second test is class name, third test is return type of variable obj, which is a object, as we are passing object as argument.

In the statement **test result;** result is variable which is used to store the value of summation of 2 objects and hence its return value is also a object and hence its return type is class name.

In the statement **result.n=n+obj.n**, obj.n refers to n value that we are passing into the object (the n value of object t2) and n refers to n value of t1 object, result.n refers to n value of object result.

In the main() method, and in the statement **t3=t1.addition(t2)**, t1 refers to invoking object and t2 refers to the parameter.

Finally we are returning temp value, like all other functions here we no need to mention the return type, just we will return the variable itself.]

Example: obj_max.cpp

```

#include<iostream.h>
#include<conio.h>
class test
{
private:
    int n;
public:
    void accept()
    {
        cin>>n;
    }
    void disp()
    {

```



```

        cout<<n<<endl;
    }
    test max(test);
};
test test::max(test t)
{
    test temp;
    if(t.n>n)
        temp.n=t.n;
    else
        temp.n=n;
    return temp;
}
void main()
{
    test t1,t2,t3;
    clrscr();
    cout<<"Enter data in object 1:";
    t1.accept();
    cout<<"Enter data in object 2: ";
    t2.accept();
    t3=t1.max(t2);
    cout<<"Maximum of 2 objects : ";
    t3.disp();
    getch();
}

```

Difference between struct and class in C++ :

In C++, the only difference between struct and class is default access specifier. In structures, the default access specifier is public, whereas in class the default access specifier is private.

Example for Structure :

```

#include<iostream.h>
#include<conio.h>

```

```

struct test
{
    int n;
    void accept()
    {
        cin>>n;
    }
    void disp()
    {
        cout<<n;
    }
};
void main()
{
    clrscr();
    test t;
    cout<<"Enter a number :";
    t.accept();
    cout<<"Given number :";
    t.disp();
    getch();
}

```

Example for class :

```

#include<iostream.h>
#include<conio.h>
struct test
{
    private :
        int n;
    public :
        void accept()

        {
            cin>>n;
        }
        void disp()
        {
            cout<<n;

```

```

    }
};
void main()
{
    clrscr();
    test t;
    cout<<"Enter a number :";
    t.accept();
    cout<<"Given number :";
    t.disp();
    getch();
}

```

Note :

If we did not use the keyword public in class example, the class will assume the methods as private because the default access specifier in class is private and we cannot access the methods with the help of an object. Whereas in struct, the default access specifier is public, hence we need not to declare the methods as public.

friend functions and classes:

friend functions :

The functions that are declared with the keyword friend are known as friend functions or friend methods. The function declaration should be preceded by the keyword friend inside the class definition, then the function is defined outside the class anywhere in the program, like normal C++ functions. The function definition does not use either keyword friend or the class name with scope resolution operator.

A function can be declared as friend in any number of classes. A friend function although not a member function has full access rights to the private members or data of its class.

Example : *friend.cpp*

```
#include<iostream.h>
```

```

#include<conio.h>
class test
{
    private:
        int n;
    public:
        friend void accept(test &)
        friend void disp(test)
};
void accept(test &a)
{
    cin>>a.n;
}
void disp(test a)
{
    cout<<a.n;
}
void main()
{
    clrscr();
    test t;
    cout<<"Enter any number : ";
    accept(t);
    cout<<"Given number is  : ";
    disp(t);
    getch();
}

```

Example : *frined1.cpp*

```

#include<iostream.h>
#include<conio.h>
class test
{
    private:
        int n;
        friend void fun();
};
void fun()
{

```

```

    test t;
    cout<<"Enter data in the object : ";
    cin>>t.n;
    cout<<"Given data in the object : "<<t.n;
}
void main()
{
    clrscr();
    fun();
    getch();
}

```

Program: *frnd_ad1.cpp*

Addition of 2 objects using friend function

```

#include<iostream.h>
#include<conio.h>
class test
{
    private:
        int n;
    public:
        void accept()
        {
            cin>>n;
        }
        void disp()
        {
            cout<<n;
        }
        friend test sum(test,test);
};
test sum(test x,test y)
{
    test r;
    r.n=x.n+y.n;
    return r;
}
void main()

```

```

{
    test t1,t2,t3;
    clrscr();
    cout<<"Enter data in object 1 : ";
    t1.accept();
    cout<<"Enter data in object 2 : ";
    t2.accept();
    t3=sum(t1,t2);
    cout<<"Addition of 2 objects  : ";
    t3.disp();
    getch();
}

```

Program : frnd_ad2.cpp

Addition of objects of 2 different classes using friend function.
 (it is possible only through friend functions)

```

#include<iostream.h>
#include<conio.h>
class class2;
class class3;
class class1
{
    private:
        int n;
    public:
        void accept()
        {
            cin>>n;
        }
        friend class3 sum(class1,class2);
};
class class2
{
    private:
        int n;
    public:
        void accept()
        {

```

```

        cin>>n;
    }
    friend class3 sum(class1,class2);
};
class class3
{
    private:
        int n;
    public:
        void disp()
        {
            cout<<n;
        }
        friend class3 sum(class1,class2);
};
class3 sum(class1 cls1,class2 cls2)
{
    class3 cls3;
    cls3.n=cls1.n+cls2.n;
    return cls3;
}
void main()
{
    class1 c1;
    class2 c2;
    class3 c3;
    clrscr();
    cout<<"Enter any data in object of class 1 : ";
    c1.accept();
    cout<<"Enter any data in object of class 2 : ";
    c2.accept();
    c3=sum(c1,c2);
    cout<<"Addition of objects of 2 different class : ";
    c3.disp();
    getch();
}

```

friend classes :

If we make the entire class as a friend, then automatically all the members of

the class become friends.

In the below example, in class cl1, we have declared the entire class cl2 as a friend, hence as a result all the member functions of the cl2 can access all the private members of class cl1.

Containership :

Containing a class object inside another class is called as containership. The class which is containing the object is called as container class.

Example : *frnd_cls.cpp_*

```
#include<iostream.h>
#include<conio.h>
class second;
class class1
{
    private:
        int n;
        friend class2;
};
class class2
{
    private:
        class1 cls1;
    public:
        /* void accept()
        {
            cout<<"Enter data in object of class1 : ";
            cin>>cls1.n;
        }
        void disp()
        {
            cout<<"Data in the object of class1  : ";
            cout<<cls1.n;
        }
        */
```



```

void access()
{
    cout<<"Enter data in object 1 : ";
    cin>>t.n;
    cout<<"Given data : " <<t.n;
}
};
void main()
{
    class2 cls2;
    clrscr();
    cls2.accept();          cls2.access();
    cls2.disp();
    getch(); }

```

Constructors and Destructors :

Constructor :

It is a method(function) that has the same name as its class, it has no return data type and is automatically executed only at the time of declaration of an object of the class. These are used to initialize the data members of its class.

Example : cons.cpp

```

#include<iostream.h>
#include<conio.h>
class test
{
    public:
    test()
    {
        cout<<"Have a Nice Day";
    }
}

```

```

    }
};
void main()
{
    clrscr();
    test t;
    // clrscr();
    getch();
}

```

Types of constructors :

C++ supports 3 types of constructors.

- 1) Non parameterized (or) default constructor.
- 2) Parameterized constructor.
- 3) Copy constructor.

Non parameterized (or)Default constructors :

A constructor that accepts no parameters is called as default constructor. It is used to construct a default object of that class. If no such constructor is defined in the class, then the compiler supplies a default constructor.

Example : cons_exp.cpp

```

#include<iostream.h>
#include<conio.h>
class test
{
    private:
        int n;
    public:
        test()
        {
            n=100;
        }
        void disp()
        {

```

```

        cout<<n;
    }
};
void main()
{
    test t;
    clrscr();
    cout<<"Data in the object : ";
    t.disp();
    getch();
}

```

Parameterized constructor :

A constructor that accepts parameters is known as parameterized constructor. We must pass the initial values as arguments to the constructor function when an object is declared. This can be done in 2 ways.

1. By calling the constructor implicitly :

Syntax : class_name object_name(arg_list);

2. By calling the constructor explicitly :

Syntax : class_name object_name = class_name (arg_list);

Example : *cons_parm.cpp*

```

#include<iostream.h>
#include<conio.h>
class test
{
    private:
        int n;
    public:
        test(int x)
        {

```

```

        n=x;
    }
    void disp()
    {
        cout<<n;
    }
};
void main()
{
    test t1(200),t2(300);
    clrscr();
    cout<<"Data in the object 1 : ";
    t1.disp();
    cout<<"\nData in the object 2 : ";
    t2.disp();
    getch();
}

```

Example : *cons_parm1.cpp*

```

#include<iostream.h>
#include<conio.h>
class num
{
    private:
        int n;
    public:
        num(int x)
        {
            n=x;
        }
        void disp()
        {
            cout<<n<<endl;
        }
};
void main()
{
    num n1=num(100); // explicit //
    num n2(100);    // implicit //
}

```

```
cout<<"Given data in the object : ";
```

Copy constructor :

A copy constructor is one which allows to create an object of given class from another object of same class. A copy constructor takes one argument that of type reference to the class.

Constructor overloading :

We can use more than one constructor in a class is known as constructor overloading.

Example : *copycons.cpp*

```
#include<iostream.h>
#include<conio.h>
class num
{
    private:
        int n;
    public:
        num(int x)
        {
            n=x;
        }
        num(num &nm)
        {
            n=nm.n;
        }
        void disp()
        {
            cout<<n<<endl;
        }
};
void main()
{
    num n1(100);
    num n2(n1);
    num n3=n1;
    cout<<"Data in the object 1 : ";
```

```

n1.disp();
cout<<"Data in the object 2 : ";
n2.disp();
cout<<"Data in the object 3 : ";
n3.disp();
getch();
}

```

Example : cons_ol.cpp

copy constructor and constructor overloading

```

#include<iostream.h>
#include<conio.h>
#include<string.h>
#include<iomanip.h>
class emp
{
    private:
        int eno;
        char ename[20];
        float esal;
    public:
        emp(int,char *,float);
        emp(emp &);
        void disp();
};
emp::emp(int no,char *name,float sal)
{
    eno=no;
    strcpy(ename,name);
    esal=sal;
}
emp::emp(emp &e)
{
    eno=e.eno;
    strcpy(ename,e.ename);
    esal=e.esal;
}
void emp::disp()

```

```

{
    cout<<endl<<endl<<"EMPNO  : "<<eno<<endl;
    cout<<"EMPNAME: "<<ename<<endl;
    cout<<"EMPSAL : "<<esal<<endl;
}
void main()
{
    emp e1(1,"RAJI",100000);
    emp e2(e1);
    clrscr();
    cout<<setprecision(2);
    cout<<setiosflags(ios::showpoint);
    cout<<setiosflags(ios::fixed);
    cout<<"Enter Employee data in object 1"<<endl;
    e1.disp();
    cout<<endl;
    cout<<"Employee data in object 2      ";
    e2.disp();
    getch();
}

```

Constructor with default arguments :

It is also possible to define constructor with default arguments.

Example : *cons_def.cpp*

```

#include<iostream.h>
#include<conio.h>
class test
{
    private:
        int n;
    public:
        test()
        {
            n=100;
        }
        test(int x)
        {

```

```

        n=x;
    }
    void disp()
    {
        cout<<n<<endl;
    }
};
void main()
{
    test t1;    // test t1,t2(200);
    test t2(200);
    clrscr();
    cout<<"Data in object 1 :";
    t1.disp();
    cout<<"Data in object 2 :";
    t2.disp();
    getch();
}

```

Invoking private constructors :

Example : *prv_cons.cpp*

```

#include<iostream.h>
#include<conio.h>
class test
{
    private:
        test()
        {
            cout<<"private construcor is invoked";
        }
    public:
        test(int)           // 2 constructor must not be the same, hence pass an integer
                             argument and give the
value of zero //
        {
            cout<<"public constructor is invoked"<<endl;
        }
}

```



```

        void func()
        {
            test t;
        }
};
void main()
{
    clrscr();
    test obj(0);
    obj.func();
    getch();
}

```

Type casting :

The type conversion from basic type to a class type is very easy to accomplish. This is done with the help of parameterized constructor.

Example : *typecast.cpp*

```

#include<iostream.h>
#include<conio.h>
class test
{
    private:
        int n;
    public:
        test(int x)
        {
            n=x;
        }
        void disp()
        {
            cout<<n;
        }
};
void main()
{
    test t1(100); // or test t1=100; // conversion of basic type to class type//
}

```

```

clrscr();
cout<<"Data in the object 1: ";
t1.disp();
getch();
}

```

This syntax is not worked in VC++

Nameless object technique :

In the returning object concept, the declaration of temporary object can be avoided, by using nameless object technique. In this technique an object can be used without name and this is done with the help of constructor.

Syntax :

```

classname(data);

```

Example : *nless_obj.cpp*

```

#include<iostream.h>
#include<conio.h>
class test
{
    private:
        int n;
    public:
        test(int x=0)
        {
            n=x;
        }
        void accept()
        {
            cin>>n;
        }
}

```

```

    void disp()
    {
        cout<<n;
    }
    test sum(test);
};
test test::sum(test t)
{
    return test(n+t.n);
}
void main()
{
    test t1,t2,t3;
    clrscr();
    cout<<"Enter data in object 1 : ";
    t1.accept();
    cout<<"Enter data in object 2 : ";
    t2.accept();
    cout<<"Addition of 2 objects  : ";
    t3=t1.sum(t2);
    t3.disp();
    getch();
}

```

nameless object

Destructor :

A destructor is a method that has the same name as its class, but preceded by a tilde mark(~). A destructor is executed at the time of destroying the object. The most important use of destructor is to deallocate the memory that was allocated for the objects by the constructor.

Note : Destructor has no return data type.

Example: destr.cpp

```

#include<iostream.h>
#include<conio.h>
class test
{

```

```

public:
    test()
    {
        cout<<"Object is created"<<endl;
    }
    ~test()
    {
        cout<<"Object is destroyed";
    }
};
void main()
{
    clrscr();
    test t;
    //getch(); // to see the output press Alt + F5 //
}

```

Dynamic memory management in C++ :

Pointer :

Pointer is a variable which stores the address of another variable. A pointer always holds the address of memory location.

Declaration :

A pointer declaration is similar to normal variable, but preceded by * .

Syntax :

datatype *identifier;

Eg :

- 1) int *p;
- 2) emp *e;

Initialization :

- 1) int n;
int *p = &n;

```
2) emp e;  
   emp *p = &e;
```

Assignment :

```
1) int n,*p;  
   p = &n;  
  
2) emp e,*p;  
   p = &e;
```

Object pointers (or) Pointer to objects :

Object pointers are useful to create objects at runtime. We can also use an object pointer to access public members of its class.

Arrow (→) :

It is used to access the members of a class with the help of object pointers.

Syntax : pointer_objectname → membername;

Eg : p → disp();

Example : ptr_obj.cpp

```
#include<conio.h>  
#include<iostream.h>  
class test  
{  
    private:  
        int n;  
    public:  
        void accept()  
        {  
            cin>>n;  
        }  
}
```

```

        void disp()
        {
            cout<<n<<endl;
        }
};
void main()
{
    test t,*p;
    clrscr();
    p=&t;
    cout<<"Enter data in object 1 : ";
    p->accept();
    cout<<"Data in the object 1    : ";
    p->disp();
    getch();
}

```

this

Dynamic memory allocation :

The process of allocating memory at runtime is known as dynamic memory allocation

new :

It is an operator which is used to create objects (or) allocate memory at runtime.

Syntax: <pointer_to_name> = new <name>[[name_initializer]];

The new operator tries to create an object by allocating sizeof <name> bytes in the heap.

Eg :

```

test *t;
t = new test;           // 1 object //
t = new test[5];        // 5 objects //

```

delete :

It is used to deallocate memory at run time.

Syntax: delete <pointer_to_name>;

The delete operator destroys the object by deallocating sizeof [<name>] bytes (pointed to by <pointer_to_name>).

Eg :

- 1) test *t;
 t=new test;
 delete t; // 1 object //

- 2) test *t;
 t=new test[5];

 delete []t; // 5 objects //

Example : dma.cpp

```
#include<iostream.h>
#include<conio.h>
class test
{
public:
    test()
    {
        cout<<"Object is created" <<endl;
    }
    ~test()
    {
        cout<<"Object is destroyed"<<endl;
    }
};
void main()
```

```

{
    clrscr();
    test *t;
    t=new test; //t=new test[4];
    delete t; // delete []t;
    getch();
}

```

Example : dma1.cpp

```

#include<iostream.h>
#include<conio.h>
class num
{
    private:
        int n;
    public:
        void accept();
        void disp();
};
void num::accept()
{
    cin>>n;
}
void num::disp()
{
    cout<<n<<endl;
}
void main()
{
    num *p;
    p=new num;
    clrscr();
    cout<<"Enter data in the object : ";
    p->accept();
    cout<<"Given data in the object : ";
    p->disp();
    delete p;
    getch();
}

```


Group of objects using dynamic memory allocation :

Example : *dma2.cpp*

```
#include<iostream.h>
#include<conio.h>
#include<iomanip.h>
class item
{
    private:
        int ino;
        char iname[20];
        float cost;
    public:
        void accept();
        void disp();
};
void item::accept()
{
    cout<<"Enter Item Number : ";
    cin>>ino;
    cout<<"Enter Item Name   : ";
    cin.ignore();
    cin.getline(iname,20);
    cout<<"Enter Item Cost   : ";
    cin>>cost;
}
void item::disp()
{
    cout<<ino<<setw(10)<<iname<<setw(15)<<cost<<endl;
}
void main()
{
    item *p;
    int n,i;
    clrscr();
    cout<<setprecision(2);
    cout<<setiosflags(ios::showpoint);
    cout<<setiosflags(ios::fixed);
```

```

cout<<"Enter number of Records : ";
cin>>n;
p=new item[n];
for(i=0;i<n;i++)
{
    cout<<endl<<"Enter Record "<<i+1<<endl<<endl;
    (p+i)->accept();
}
clrscr();
cout<<"ITEM NO"<<setw(10)<<"ITEM NAME"<<setw(15)<<
    "ITEM COST"<<endl;
cout<<"-----"<<endl<<endl;
for(i=0;i<n;i++)
{
    (p+i)->disp();
}
delete []p;
getch();
}

```

Example : dma3.cpp

```

#include<iostream.h>
#include<conio.h>
#include<iomanip.h>
class emp
{
    private:
        int eno;
        char ename[20];
        float esal;
    public:
        void accept();
        void disp();
};
void emp::accept()
{
    cout<<"Enter Employee Number    : ";
    cin>>eno;
    cout<<"Enter Employee Name      : ";

```

```

    cin.ignore();
    cin.getline(ename,20);
    cout<<"Enter Employee Salarary : ";
    cin>>esal;
}
void emp::disp()
{
    cout<<eno<<setw(15)<<ename<<setw(15)<<esal<<endl;
}
void main()
{
    emp *e;
    int n,i;
    clrscr();
    cout<<setprecision(2);
    cout<<setiosflags(ios::showpoint);
    cout<<setiosflags(ios::fixed);
    cout<<"Enter number of records : ";
    cin>>n;
    e=new emp[n];
    for(i=0;i<n;i++)
    {
        cout<<endl<<"Enter Record No : "<<i+1<<endl;
        (e+i)->accept();
    }
    clrscr();
    cout<<"Employ Number"<<setw(10)<<
        "Employ Name"<<setw(10)<<"Employ Salary"<<endl;
    for(i=0;i<n;i++)
    {
        (e+i)->disp();
    }
    getch();
}

```

Example : dma4.cpp

```

#include<iostream.h>
#include<conio.h>
#include<iomanip.h>

```

```

#include<string.h>
class student
{
    private:
        int sno,c,cpp,java,tot;
        char sname[20],res[10],div[15];
        float avg;
    public:
        void accept();
        void cal();
        void disp();
};

void student::accept()
{
    cout<<"Enter Student Number : ";
    cin>>sno;
    cout<<"Enter Student Name   : ";
    cin.ignore();
    cin.getline(sname,20);
    cout<<"Enter marks in C       : ";
    cin>>c;
    cout<<"Enter marks in CPP    : ";
    cin>>cpp;
    cout<<"Enter marks in Java   : ";
    cin>>java;
}

void student::cal()
{
    tot=c+cpp+java;
    avg=(float)tot/3;
    if(c>=50 && cpp>=50 && java>=50)
    {
        strcpy(res,"PASS");
        if(avg>=60)
            strcpy(div,"FIRST");
        else
            strcpy(div,"SECOND");
    }
    else
    {

```

```

        strcpy(res,"FAIL");
        strcpy(div,"NO DIVISION");
    }
}
void student ::disp()
{
    cout<<"Marks in C      : "<<c<<<endl;
    cout<<"Marks in CPP    : "<<cpp<<<endl;
    cout<<"Marks in Java   : "<<java<<<endl;
    cout<<"Total Marks     : "<<tot<<<endl;
    cout<<"Average Marks   : "<<avg<<<endl;
    cout<<"Result          : "<<res<<<endl;
    cout<<"Division        : "<<div<<<endl<<<endl;
}
void main()
{
    student *s[100];    // array of pointers //
    char ch;
    int id=0,i;
    clrscr();
    cout<<setprecision(2);
    cout<<setiosflags(ios::showpoint);
    do
    {
        clrscr();
        cout<<"Enter Record : "<<id+1<<<endl<<<endl;
        s[id]->accept();
        s[id]->cal();
        id++;
        cout<<<endl<<"Do u want to continue(y/n) : ";
        cin>>ch;
    }
    while(ch=='y' && id<100);
    clrscr();
    for(i=0;i<id;i++)
    {
        clrscr();
        cout<<"Record "<<i+1<<<endl<<<endl;
        s[i]->disp();
        delete s[i];
    }
}

```

```

        cout<<"Press any key to continue....."<<endl<<endl;
        getch();
    }
}

```

Here student *s[100] is a group of pointers. i.e. here we are creating group of pointers rather than group of objects. If we create group of objects there is a loss of memory. i.e. if we create 100 objects, 100 * (memory for integers data types for sno, c, cpp, java, tot and char data type for sname, res, div and float data type for avg), hence there is lot of wastage of memory. If we create object pointers, we need memory of 100*(2 bytes for pointer) = 200 bytes.

Dynamic one dimensional array or Dynamic 1D Vector :

Example : dyn_1dar.cpp

```

#include<iostream.h>
#include<conio.h>
#include<iomanip.h>
class sdarr
{
private:
    int n,*a;
public:
    sdarr(int);
    ~sdarr();           runtime pointer array
    void accept();
    void disp();
};
sdarr::sdarr(int s)
{
    n=s;
    a=new int[n];
}
void sdarr::accept()
{
    for(int i=0;i<n;i++)
    {
        cin>>*(a+i);
    }
}

```

```

}
void sdarr::disp()
{
    for(int i=0;i<n;i++)
    {
        cout<<*(a+i)<<setw(5);
    }
}
sdarr::~sdarr()
{
    delete []a;
}
void main()
{
    clrscr();
    sdarr *s;
    int n;
    cout<<"Enter number of elements : ";
    cin>>n;
    s=new sdarr(n);
    cout<<"Enter array elements      : ";
    s->accept();
    cout<<"Given array elements      : ";
    s->disp();
    delete s;
    getch();
}

```

Dynamic two dimensional array :

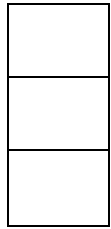
```

int **a,r,c;
(Let r=3,c=3)
a=new int *[r];
for(int i=0;i<r;i++)
{
    *(a+i)=new int[c];
}

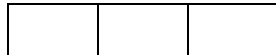
```



columns



a



rows



for each row, there are 3 columns. To get the address and value of memory location, we should use the following syntax.

Address : _

In C : $*(a+i)+j$ (To read)

In CPP : $*(*(a+i)+j)$ (To read)

Value :

In C : $*(*(a+i)+j)$ (To print)

In CPP : $*(*(a+i)+j)$ (To print)

Example : *tdarr.cpp*

```
#include<iostream.h>
#include<conio.h>
#include<iomanip.h>
class tdarr
{
private:
    int m,n,**a;
```



```

public:
    tdarr(int,int);
    ~tdarr();
    void accept();
    void disp();
};

tdarr::tdarr(int r,int c)
{
    m=r;
    n=c;
    *a=new int[m];    // a=new int *[m];
    for(int i=0;i<m;i++)
    {
        *(a+i)=new int[n];
    }
}

void tdarr::accept()
{
    for(int i=0;i<m;i++)
    {
        for(int j=0;j<n;j++)
        {
            cin>>*(*(a+i)+j);
        }
    }
}

void tdarr::disp()
{
    for(int i=0;i<m;i++)
    {
        for(int j=0;j<n;j++)
        {
            cout<<setw(5)<<*(*(a+i)+j);
        }
        cout<<endl;
    }
}

tdarr::~~tdarr()
{
    delete []a;
}

```

```

}
void main()
{
    tdarr *t;
    int m,n;
    clrscr();
    cout<<"Enter number of rows and columns : ";
    cin>>m>>n;
    t=new tdarr(m,n);
    cout<<"Enter array elements          : "<<endl;
    t->accept();
    cout<<"Display array elements        : " <<endl;
    t->disp();
    delete t;
    getch();
}

```

Containership :

Containing a class object inside another class is called as containership, and the class which contains the object is called the container class.

ASCII values :

Alt + 196 :

Alt + 179 :

191(ec,sr)

218 (sc,sr)

Alt + 218 :

Alt + 191 :

Alt + 192 :

Alt + 217 :
219(ec,er)

292 (sc,er)

Program : Design.h (creation of header file)

```
#include<iostream.h>
#include<conio.h>
class line
{
    public:
        void vline(int,int,int);
        void hline(int,int,int);
};
void line::vline(int c,int sr,int er)
{
    for(int i=sr;i<er;i++)
    {
        gotoxy(c,i);
        cout<<(char)179;
    }
}
void line::hline(int r,int sc,int ec)
{
    for(int i=sc;i<ec;i++)
    {
        gotoxy(i,r);
        cout<<(char)196;
    }
}
class box
class
{
    public:
        line l;
        another class)
        void drawbox(int,int,int,int);
};
void box::drawbox(int sc,int sr,int ec,int er)
{
    gotoxy(sc,sr);
    cout<<(char)218;
```

containership

containership object(object of

```

gotoxy(ec,sr);
cout<<(char)191;
gotoxy(sc,er);
cout<<(char)192;
gotoxy(ec,er);
cout<<(char)217;
l.hline(sr,sc+1,ec-1);
l.hline(er,sc+1,ec-1);
l.vline(sc,sr+1,er-1);
l.vline(ec,sr+1,er-1);
}

```

Program : drawbox.CPP (using the created header file)

```

#include"design.h"
#include<iostream.h>
#include<conio.h>
void main()
{
    line l;
    box b;
    clrscr();
    l.hline(15,10,20);
    l.vline(15,20,30);
    b.drawbox(5,5,75,40);
    b.drawbox(30,15,50,30);
    getch();
}

```

Constant member function :

If a member function doesnot alter any data in the class, we may declare it as a constant member function.The keyword const is appended to the function prototype in both declaration and definition. The compiler will generate an error message if such functions try to alter the data values.

Example : const_mfn.cpp

Example for constant member function

```
#include<iostream.h>
#include<conio.h>
class num
{
    private:
        int n;
    public:
        num(int x)
        {
            n=x;
        }
        void disp()
        {
            cout<<n<<endl;
        }
        void setdata(int k) const
        {
            // n=k;   cannot be modified //
        }
};
void main()
{
    clrscr();
    num nm(100);
    cout<<"Initial data in the object  :";
    nm.disp();
    nm.setdata(200);
    cout<<"Modified data in the object  :";
    nm.disp();
    getch();
}
```

constant objects :

We may create and use constant objects using ***const*** keyword before object declaration. A constant object can call only constant member functions . Whenever constant objects try to invoke non constant member function, the compiler will generate an error.

Example : const_obj.cpp

```
#include<iostream.h>
#include<conio.h>
class num
{
    private:
        int n;
    public:
        num(int x)
        {
            n=x;
        }
        void disp() const
        {
            cout<<n<<endl;
        }
        void setdata(int k) const
        {
            n=k;
        }
};
void main()
{
    const num nm(100);
    cout<<"Initial Value : ";
    nm.disp();
    cout<<"Changed Value : ";
    nm.disp();
}
```

Mutable:

We know that a class object or a member function may be declared as const,

we cannot modify any data in the class. However in some situations we want to create constant object or constant member function, but we would like to modify particular data item. In such situation we can make that particular data item as mutable.

Example : mutable1.cpp

```
#include<iostream.h>
#include<conio.h>
class test
{
    private:
        mutable int n;
    public:
        void accept()
        {
            cin>>n;
        }
        void disp() const
        {
            cout<<n<<endl;
        }
        void modify() const
        {
            cin>>n;
        }
};
void main()
{
    // clrscr();   not worked in VC++ or C-Free
    test t;
    cout<<"Enter data in the object  : ";
    t.accept();
    cout<<"Initial data in the object : ";
    t.disp();
    cout<<"Modify the data in object : ";
    t.modify();
    cout<<"Modified data in object   : ";
    t.disp();
}
```

Example : mutable2.cpp

```
#include<iostream.h>
#include<conio.h>
class num
{
    private:
        int n;
    public:
        num(int x)
        {
            n=x;
        }
        void disp() const
        {
            cout<<n<<endl;
        }
        void setdata(int k) const
        {
            // n=k;  cannot be modified //
        }
};
void main()
{
    clrscr();
    const num nm(100);
    cout<<"Initial data in the object  : ";
    nm.disp();
    nm.setdata(200);
    cout<<"Modified data in the object : ";
    nm.disp();
    getch();
}
```

explicit :

The ***explicit*** keyword is used to declare a class constructor to be a explicit constructor. In the typecasting program, we have seen one argument constructor

performs implicit conversion, since the conversion is automatic, we need not apply any casting. In case we donot want such automatic conversion to take place, we may do so by declaring constructor as ***explicit***.

Example : expl_con.cpp

```
#include<iostream.h>
#include<conio.h>
class test
{
    private:
        int n;

    public:
        explicit test(int x)
        {
            n=x;
        }
        void disp()
        {
            cout<<n;
        }
};
void main()
{
    // test t=100;    is not possible //
    test t(100);
    cout<<"Data in the object : ";
    t.disp();
}
```

Operator Overloading

It provides a flexible option of the creating of new definition for most of CPP operators. We can almost create a new language of our own by the creative use of the function and operator overloading technique.

Defining Operator overloading :

To define an additional task to an operator we must specify what it means in relation to the class to which the operator is apply. This is done with the help of special function called operator function which describes the task.

Syntax :

dec:

```
return_type    operator  symbol ([argument_list]);
```

def:

```
return_type    class_name :: operator  symbol ([argument_list])
{
    statements;
}
```

Example : oload_ad.cpp

Addition of 2 objects using Operator Overloading

```
#include<iostream.h>
#include<conio.h>
class test
{
    private:
        int n;
    public:
        void accept()
        {
            cin>>n;
        }
        void disp()
        {
            cout<<n<<endl;
        }
        test operator + (test);
}
```

```

};
test test::operator + (test s)
{
    test temp;
    temp.n=n+s.n;
    return temp;
}
void main()
{
    test t1,t2,t3;
    clrscr();
    cout<<"Enter data in object 1 : ";
    t1.accept();
    cout<<"Enter data in object 2 : ";
    t2.accept();
    cout<<"Addition of 2 objects  : ";
    t3=t1+t2;
    t3.disp();
    getch();
}

```

Rules for Operator overloading :

Although it looks simple to redefine the operators, there are certain restrictions and limitations in overloading them. Some of them are :

- 1 Only existing operators can be overloaded, new operators cannot be created.
- 2 The overloaded operators must have atleast one operand, that is of user defined data type (i.e. class type).
- 3 We cannot change the basic meaning of an operator.
- 4 Overloaded operators follows the syntax rules of the original operators.

- 5 There are some operators that cannot be overloaded.
they are sizeof, ::, ?:, ., .*
- 6 We cannot use friend functions to overload certain operators. However member functions can be used to overload them.
they are =, (), [], →
- 7 Binary operators overloaded through a member function take one explicit argument and those which are overloaded through a friend function to take two explicit arguments.
- 8 When using Binary operators overloaded through a member function, the left hand operand must be an object of the relevant class.

Example: 5_oload.cpp

Overloading of all the 5 arithmetic operators

```
#include<iostream.h>
#include<conio.h>
class test
{
private:
    int n;
public:
    void accept()
```

```

    {
        cin>>n;
    }
    void disp()
    {
        cout<<n<<endl;
    }
    test operator + (test);
    test operator - (test);
    test operator * (test);
    test operator / (test);
    test operator % (test);
};
test test::operator + (test s)
{
    test temp;
    temp.n=n+s.n;
    return temp;
}
test test::operator - (test s)
{
    test temp;
    temp.n=n-s.n;
    return temp;
}
test test::operator * (test s)
{
    test temp;
    temp.n=n*s.n;
    return temp;
}
test test::operator / (test s)
{
    test temp;
    temp.n=n/s.n;
    return temp;
}
test test::operator % (test s)
{
    test temp;

```

```

    temp.n=n%s.n;
    return temp;
}
void main()
{
    test t1,t2,t3;
    clrscr();
    cout<<"Enter data in object 1      : ";
    t1.accept();
    cout<<"Enter data in object 2      : ";
    t2.accept();
    cout<<"Addition of 2 objects        : ";
    t3=t1+t2;
    t3.disp();
    cout<<"Subtraction of 2 objects     : ";
    t3=t1-t2;
    t3.disp();
    cout<<"Multiplication of 2 objects  : ";
    t3=t1*t2;
    t3.disp();
    cout<<"Division of 2 objects         : ";
    t3=t1/t2;
    t3.disp();
    cout<<"Modulus of 2 objects          : ";
    t3=t1%t2;
    t3.disp();
    getch();
}

```

Example : (str_add.cpp)

Addition of 2 string objects

```

#include<iostream.h>
#include<conio.h>
#include<string.h>
class string
{
    private:
        char st[20];

```

```

public:
    void disp()
    {
        cout<<st<<endl;
    }
    void operator = (char *);           // (char [])
    string operator + (string);
};
void string::operator = (char *st1)    // (char st1[])
{
    strcpy(st,st1);
}
string string::operator + (string st2)
{
    string temp;
    strcpy(temp.st,st);
    strcat(temp.st,st2.st);
    return temp;
}
void main()
{
    string s1,s2,s3;
    clrscr();
    s1="WEL";
    s2="COME";
    s3=s1+s2;
    cout<<"s1 = ";
    s1.disp();
    cout<<"s2 = ";
    s2.disp();
    cout<<"s3 = ";
    s3.disp();
    getch();
}

```

Example : ovr_frnd.cpp

overloading of all the 5 arithmetic operators using friend function

```
#include<iostream.h>
```

```

#include<conio.h>
class test
{
    private:
        int n;
    public:
        void accept()
        {
            cin>>n;
        }
        void disp()
        {
            cout<<n<<endl;
        }
        friend test operator + (test,test);
        friend test operator - (test,test);
        friend test operator * (test,test);
        friend test operator / (test,test);
        friend test operator % (test,test);
};

test operator + (test x,test y)
{
    test temp;
    temp.n=x.n+y.n;
    return temp;
}

test operator - (test x,test y)
{
    test temp;
    temp.n=x.n-y.n;
    return temp;
}

test operator * (test x,test y)
{
    test temp;
    temp.n=x.n*y.n;
    return temp;
}

test operator / (test x,test y)
{

```



```

    test temp;
    temp.n=x.n/y.n;
    return temp;
}
test operator % (test x,test y)
{
    test temp;
    temp.n=x.n%y.n;
    return temp;
}
void main()
{
    test t1,t2,t3;
    clrscr();
    cout<<"Enter data in object 1      : ";
    t1.accept();
    cout<<"Enter data in object 2      : ";
    t2.accept();
    cout<<"Addition of 2 objects        : ";
    t3=t1+t2;
    t3.disp();
    cout<<"Subtraction of 2 objects     : ";
    t3=t1-t2;
    t3.disp();
    cout<<"Multiplication of 2 objects   : ";
    t3=t1*t2;
    t3.disp();
    cout<<"Division of 2 objects          : ";
    t3=t1/t2;
    t3.disp();
    cout<<"Modulus of 2 objects           : ";
    t3=t1%t2;
    t3.disp();
    getch();
}

```

Example : *expr_ol.cpp*

***To evaluate the following expression using operator over loading.
 $b=a+100$, where a and b are objects***

```

#include<iostream.h>
#include<conio.h>
class test
{
    private:
        int n;
    public:
        void accept()
        {
            cin>>n;
        }
        void disp()
        {
            cout<<n<<endl;
        }
        test operator + (int);
};
test:: test operator + (int x)
{
    test temp;
    temp.n=n+x;
    return temp;
}
void main()
{
    test a,b;
    clrscr();
    cout<<"Enter data in object A : ";
    a.accept();
    b=a+100;
    cout<<"B = A + 100          : ";
    b.disp();
    getch();
}

```

Example : unary_ol.cpp

To overload all Unary operators

```

#include<iostream.h>
#include<conio.h>
class test
{
    private:
        int n;
    public:
        void accept()
        {
            cin>>n;
        }
        void disp()
        {
            cout<<n<<endl;
        }
        void operator - ()
        {
            n=-n;
        }
        void operator ++()
        {
            n=++n; //++n;
        }
        void operator ++(int)
        {
            n=n+1; //n++
        }
        void operator --()
        {
            n=--n; //--n;
        }
        void operator -- (int )
        {
            n=n-1; //n--
        }
};
void main()
{
    test t;
    clrscr();

```

```

    cout<<"Enter any data in the object T : ";
    t.accept();
    -t;
//  ++t;
//  t++;
//  --t;
//  t--;
    cout<<"Data in the object          : ";
    t.disp();
    getch();
}

```

Example : unaryol2.cpp

To overload all Unary operators with return value

```

#include<iostream.h>
#include<conio.h>
class test
{
    private:
        int n;
    public:
        void accept()
        {
            cin>>n;
        }
        void disp()
        {
            cout<<n<<endl;
        }
        test operator - ()
        {
            test temp;
            temp.n=-n;
            return temp;
        }
        test operator ++()

```

```

    {
        test temp;
        temp.n=++n; //++n;
        return temp;
    }
test operator ++(int)
{
    test temp;
    temp.n=n+1; //n++
    return temp;
}
test operator --()
{
    test temp;
    temp.n=--n; //--n;
    return temp;
}
test operator -- (int )
{
    test temp;
    temp.n=n-1; //n--
    return temp;
}
};
void main()
{
    test t;
    clrscr();
    cout<<"Enter any data in the object T : ";
    t.accept();
    t=-t;
    cout<<"Data in the object          : ";
    t.disp();
    cout<<"Enter any data in the object T : ";
    t.accept();
    t=++t;
    cout<<"Data in the object          : ";
    t.disp();
    cout<<"Enter any data in the object T : ";
    t.accept();

```

```

t=t++;
cout<<"Data in the object          : ";
t.disp();
cout<<"Enter any data in the object T : ";
t.accept();
t=--t;
cout<<"Data in the object          : ";
t.disp();
cout<<"Enter any data in the object T : ";
t.accept();
t=t--;
cout<<"Data in the object          : ";
t.disp();
}

```

this :

It is a predefined pointer which points to the object itself. It stores the address of the object itself.

Example : *this1.cp*

To find maximum of 2 objects using this

```

#include<iostream.h>
#include<conio.h>
class test
{
private:
    int n;
public:
    void accept()
    {
        cin>>n;           // cin>>this->n (efficient way)
    }
    void disp()
    {
        cout<<n<<endl;    // cout<<this->n (efficient way)
    }
    test maximum(test);
};

```

```

test test :: maximum(test t)
{
    if(n>t.n)
        return *this;
    else
        return t;
}
void main()
{
    test t1,t2,max;
    clrscr();
    cout<<"Enter data in object 1 : ";
    t1.accept();
    cout<<"Enter data in object 2 : ";
    t2.accept();
    max=t1.maximum(t2);
    cout<<"Maximum of 2 objects    : ";
    max.disp();
    getch();
}

```

Example : *arrow_ol.cpp*

To overload arrow operator

```

#include<iostream.h>
#include<conio.h>
class test
{
    private:
        int n;
    public:
        void accept()
        {
            cin>>n;
        }
        void disp()
        {
            cout<<n<<endl;
        }
}

```

```

        test* operator -> ();
};
test* test::operator -> ()
{
    return this;
}
void main()
{
    test t;
    cout<<"Enter data in object : ";
    t->accept();
    cout<<"Given data in object : ";
    t->disp();
}

```

We should use -> operator for pointer objects calling. But here object **t** is not a pointer. Hence the return type of object must be pointer of type test, i.e. **test ***. **this** is the keyword which returns the address of the object itself._

Pointer to data mebers :

It is possible to take the address of data member of a class and assign it to a pointer outside the address of a data member obtained by applying the operator & (address) to a fully qualified class data member name. A class member pointer can be declare using the operator ::* the class name and it can be accessed with the help of operator .* with the object.

Syntax :

```
datatype class_name :: *identifier = &class_name :: datamember_name;
```

Eg : int num :: *p = &num :: n;

Example : ptrdm.cpp

```

#include<iostream.h>
#include<conio.h>
class num
{
    public:
        int n;

```



```

};
void main()
{
    int num::*p=&num::n;
    num nb;
    clrscr();
    nb.*p=10;
    cout<<"Value of n : "<<nb.n<<endl;
    nb.n=20;
    cout<<"Value of p : "<<nb.*p<<endl;
    getch();
}

```

Example : *equal_ol.cpp*

Overloading ==,>,<,>=,<= operators

```

#include<iostream.h>
#include<conio.h>
class test
{
    private:
        int n;
    public:
        void accept()
        {
            cin>>n;
        }
        int operator ==(test);
        int operator > (test);
        int operator < (test);
};
int test::operator ==(test t)
{
    if(n==t.n)
        return 1;
    else
        return 0;
}

```

```

int test::operator > (test t)
{
    if(n>t.n)
        return 1;
    else
        return 0;
}
int test::operator < (test t)
{
    if(n<t.n)
        return 1;
    else
        return 0;
}
void main()
{
    test t1,t2;
    clrscr();
    cout<<"Enter data in object 1 :";
    t1.accept();
    cout<<"Enter data in object 2 :";
    t2.accept();
    if(t1==t2)
        cout<<"Object-1 and Object-2 are equal";
    if(t1>t2)
        cout<<"Object-1 is greate than Object-2";
    if(t1<t2)
        cout<<"Object-1 is less than Object-2";
    getch();
}

```

Example : ins_ext.cpp

To overload insertion and extraction operators

```

#include<iostream.h>
#include<conio.h>
class num
{
    private:

```

```

        int n;
    public:
        friend istream & operator >> (istream &,num &);
        friend ostream & operator << (ostream &,num &);
};
istream & operator >> (istream &is,num &ob)
{
    is>>ob.n;
    return is;
}
ostream & operator << (ostream &os,num &ob)
{
    os<<ob.n;
    return os;
}
void main()
{
    num nm;
    clrscr();
    cout<<"Enter any value in object : ";
    cin>>nm;
    cout<<"Given value in the object : ";
    cout<<nm;
    getch();
}

```

To overload an operator, the left hand argument must be of class type. But here the left hand argument is not of class type, they are input and output streams (cin>>nm and cout<<nm). Hence it is not possible to overload the operator with the member function. Hence we should use friend function to overload the operator, because in friend functions we are passing 2 arguments as parameters. Here we are passing input stream reference and class reference as parameters. The return type of input and output stream is reference of that type. The return type void is also acceptable, but it is not much efficient.

Example : *ins_extr.cpp*

```

#include<iostream.h>
#include<conio.h>
#include<iomanip.h>

```

```

class emp
{
    private:
        int eno;
        char ename[20];
        float esal;
    public:
        friend istream& operator >> (istream &,emp &);
        friend ostream& operator << (ostream &,emp &);
};

istream& operator >> (istream &i,emp &e)
{
    cout<<"Enter Emp Number : ";
    i>>e.eno;
    cout<<"Enter Emp Name   : ";
    i.ignore();
    i.getline(e.ename,20);
    cout<<"Enter Emp salary : ";
    i>>e.esal;
    return i;
}

ostream& operator << (ostream &o,emp &e)
{
    o<<setprecision(2);
    o<<setiosflags(ios::showpoint);
    o<<setiosflags(ios::fixed);
    o<<endl<<"ENO   : "<<e.eno<<endl;
    o<<"ENAME : "<<e.ename<<endl;
    o<<"ESAL  : "<<e.esal<<endl;
    return o;
}

void main()
{
    emp e;
    clrscr();
    cout<<"Enter Employ data"<<endl<<endl;
    cin>>e;
    cout<<endl<<"Given Employ data"<<endl;
    cout<<e;
    getch();
}

```

```
}
```

Example : *sub_parn.cpp*

To overload sub script and paranthesis ([],0)

```
#include<iostream.h>
#include<conio.h>
#include<string.h>
class test
{
    private:
        int n;
        char st[20];
    public:
        void operator [] (int);
        void operator () (char[]);
};
void test::operator [] (int k)
{
    n=k;
    cout<<n<<endl;
}
void test::operator () (char st1[])
{
    strcpy(st,st1);
    cout<<st;
}
void main()
{
    test t;
    clrscr();
    cout<<"Given number : ";
    t[100];
    cout<<"Given string : ";
    t("RAJI");
    getch();
}
```

Example : *new_del.cpp*

Overloading new and delete operators

```
#include<iostream.h>
#include<conio.h>
#include<alloc.h>
class test
{
    public:
        test()
        {
            cout<<"Constructor is invoked"<<endl;
        }
        ~test()
        {
            cout<<"Destructor is invoked"<<endl;
        }
        void * operator new(size_t);
        void operator delete(void *);
};
void * test::operator new(size_t sz)
{
    test *p;
    p=(test *)malloc(sz);
    cout<<"Object is created"<<endl;
    return p;
}
void test::operator delete(void *p)
{
    free(p);    //delete p;
    cout<<"Object is destroyed"<<endl;
}
void main()
{
    clrscr();
    test *t;
    t=new test;
    delete t;
    getch();
}
```

Example : mat_add.cpp

Matrix addition through operator overloading

```
#include<iostream.h>
#include<conio.h>
#include<iomanip.h>
class matrix
{
    private:
        int a[5][5],r,c;
    public:
        matrix(int m,int n)
        {
            r=m;
            c=n;
        }
        friend istream & operator >> (istream &,matrix &);
        friend ostream & operator << (ostream &,matrix &);
        friend matrix operator + (matrix,matrix);
};

istream & operator >> (istream & is,matrix & m)
{
    for(int i=0;i<m.r;i++)
    {
        for(int j=0;j<m.c;j++)
        {
            is>>m.a[i][j];
        }
    }
    return is;
}

ostream & operator << (ostream & os,matrix & m)
{
    for(int i=0;i<m.r;i++)
    {
        for(int j=0;j<m.c;j++)
        {
            os<<setw(3)<<m.a[i][j];
```

```

    }
    os<<endl;
}
return os;
}
matrix operator + (matrix m1,matrix m2)
{
    matrix temp(m1.r,m1.c);
    for(int i=0;i<m1.r;i++)
    {
        for(int j=0;j<m1.c;j++)
        {
            temp.a[i][j]=m1.a[i][j]+m2.a[i][j];
        }
    }
    return temp;
}
void main()
{
    int m,n;
    clrscr();
    cout<<"Enter number of rows and columns : ";
    cin>>m>>n;
    matrix a(m,n),b(m,n),c(m,n);
    cout<<"Enter First Matrix  : "<<endl;
    cin>>a;
    cout<<"Enter Second Matrix : "<<endl;
    cin>>b;
    c=a+b;
    cout<<"Addition of 2 matrices : "<<endl;
    cout<<c;
    getch();
}

```

Example : mat_mul.cpp

Matrix Multiplication through operator overloading

```

#include<iostream.h>
#include<conio.h>

```



```

#include<iomanip.h>
#include<process.h>
class matrix
{
private:
    int a[5][5],r,c;
public:
    matrix(int m,int n)
    {
        r=m;
        c=n;
    }
    friend istream & operator >> (istream &,matrix &);
    friend ostream & operator << (ostream &,matrix &);
    friend matrix operator * (matrix,matrix);
};

istream & operator >> (istream & is,matrix & m)
{
    for(int i=0;i<m.r;i++)
    {
        for(int j=0;j<m.c;j++)
        {
            is>>m.a[i][j];
        }
    }
    return is;
}

ostream & operator << (ostream & os,matrix & m)
{
    for(int i=0;i<m.r;i++)
    {
        for(int j=0;j<m.c;j++)
        {
            os<<setw(3)<<m.a[i][j];
        }
        os<<endl;
    }
    return os;
}

matrix operator * (matrix m1,matrix m2)

```

```

{
    matrix temp(m1.r,m2.c);
    for(int i=0;i<m1.r;i++)
    {
        for(int j=0;j<m2.c;j++)
        {
            temp.a[i][j]=0;
            for(int k=0;k<m1.c;k++)
            {
                temp.a[i][j]=temp.a[i][j]+(m1.a[i][k]*m2.a[k][j]);
            }
        }
    }
    return temp;
}

void main()
{
    int m,n,p,q;
    clrscr();
    cout<<"Enter First Matrix rows and columns  : ";
    cin>>m>>n;
    cout<<"Enter Second Matrix rows and columns : ";
    cin>>p>>q;
    if(n!=p)
    {
        cout<<"Matrix Multiplication is not Possible";
        getch();
        exit(0);
    }
    matrix a(m,n),b(p,q),c(m,q);
    cout<<"Enter First Matrix  : "<<endl;
    cin>>a;
    cout<<"Enter Second Matrix : "<<endl;
    cin>>b;
    c=a*b;
    cout<<"Multiplication of 2 matrices : "<<endl;
    cout<<c;
    getch();
}

```

Type casting :

1. Basic type to class type :

The conversion from basic type to class type is easy to accomplish, this is done with the help of constructor.

conversion of basic type to class type

```
#include<iostream.h>
#include<conio.h>
class test
{
    private:
        int n;
    public:
        test(int x)
        {
            n=x;
        }
        void disp()
        {
            cout<<n;
        }
};
void main()
{
    test t1(100); // or test t1=100; // conversion of basic type to class type//
    clrscr();
    cout<<"Data in the object 1: ";
    t1.disp();
    getch();
}
```

2. Class type to basic type :

The constructor did a fine job in type conversion from a basic type to a class type. But the constructor function doesnot convert from class type to a basic type. C++ allows us to define an overloaded casting operator, that could be used to convert a class type data to a basic type.

Syntax :

The general form of an overloaded casting operator function is :

```
operator type_name()
{
    statements;
}
```

The casting operator function should satisfied the following conditions :

1. It must be a class member.
2. It must not specify any return type, but it returns a value specified type name.
3. It must not have any arguments.
4. We should write this syntax within the class definition.

Example : (cl2basic.cpp)

Conversion of class type to basic type

```
#include<iostream.h>
#include<conio.h>
class test
{
    private:
        int n;
    public:
        test(int x)
        {
            n=x;
        }
        void disp()
        {
            cout<<n;
        }
}
```

```

        operator int()
        {
            return n;
        }
};
void main()
{
    int n;
    clrscr();
    test t=100; // basic type to class type //
    cout<<"Data in the object : ";
    t.disp();
    n=t; // class type to basic type //
    cout<<"n : "+n;
    getch();
}

```

One class type to another :

We have just seen data conversion techniques from basic type to class type and class type to basic type. But there are situations where we would have to convert one class type to another class type data. This is also be done with the help of casting operator function.

Example : (cls2cls.cpp)

Conversion of one class type to another class type

```

#include<iostream.h>
#include<conio.h>
class num; // sometimes it is necessary //
class test
{
    private:
        int n;
    public:
        void disp()
        {
            cout<<n<<endl;

```

```

    }
    friend num;
};
class num
{
    private:
        int n;
    public:
        void accept()
        {
            cin>>n;
        }
        void disp()
        {
            cout<<n<<endl;
        }
        operator test()
        {
            test obj;
            obj.n=n;
            return obj;
        }
};
void main()
{
    test t;
    num nm;
    cout<<"Enter data in class num      : ";
    nm.accept();
    cout<<"Given data in class num      : ";
    nm.disp();
    t=nm;
    cout<<"Initialized data in class test : ";
    t.disp();
}

```

Inheritance

It is a mechanism which is used to extend the definition of existing

class. The extension can be done by declaring some other class. The class that is originally present is called as base class and the class which is extended with the help of inheritance is called as derived class. A derived class always can access the members of base class, whereas a base class never access the members of derived class. The main purpose of inheritance is ***reusability*** of definition that is already made.

Defining a derived class :

```
class derived_class : [visibility mode]    base_class
{
    definition;
};
```

Visibility modes are 3 types.
They are :
1. public
2. private
3. protected
Default Visibility mode is Private.

Protected :

A member of a class is declared as protected, it is accessible by the member functions within its class and any other class derived from it. It cannot be accessed by the functions outside of these two classes.

Private inheritance (or) visibility mode is private :

When a base class is privately inherited, the public and protected members of base class become private members of derived class.

Note : Private class members will not inherited

Public inheritance or visibility mode is public :

Base class

Not inherited

Public inheritance

Derived class

Protected area :

Public area :

When a base class is publicly inherited, the public members of base class become public members of derived class and the protected members of base class become protected members of derived class.

Protected inheritance or visibility mode is protected :

When a base class is protectedly inherited, the public and protected members of base class become protected members of derived class.

Base class

Not inherited

Protected inheritance

Derived class

Protected area :

C++ supports 5 types of inheritance

1. Single (or) Simple inheritance
2. Multiple inheritance
3. Hierarchical inheritance
4. Multi level inheritance
5. Hybrid inheritance

Single or Simple inheritance :

A derived class with only one base class is called as Single inheritance.

Syntax :

```

class derv_class : [ visibility_mode] base_class
{
    Class body;
};

```

Example : *inh1.cpp*

```

#include<iostream.h>
#include<conio.h>
class basecls
{
    private:
        int n;
    public:
        void accept()
        {
            cin>>n;
        }
        void disp()
        {
            cout<<n;
        }
};
class dercls:public basecls
{
};
void main()
{
    dercls d;
    clrscr();
    cout<<"Enter data in derived class : ";
    d.accept();
    cout<<"Data in the derived class : ";
    d.disp();
    getch();
}

```

Example : *inh2.cpp*

```

#include<iostream.h>
#include<conio.h>
class basecls
{
    private:
        int n;
    public:
        void accept()
        {
            cin>>n;
        }
        void disp()
        {
            cout<<n;
        }
};
class dercls : basecls
{
    public:
        void input()                default visibility mode is private
        {
            accept();
        }
        void output()
        {
            disp();
        }
};
void main()
{
    dercls d;
    clrscr();
    cout<<"Enter data in derived class : ";
    d.input();
    cout<<"Data in the derived class    : ";
    d.output();
}

```

Overriding :

If a members are declared with the same name both in base class and derived class, a derived class object always give preference to the derived class member by overlooking base class member. Hence it is called overriding base class member.

Invoking the overriding member by the following way :

```
class_name :: function_name([arg_list]);  
class_name :: datamember;
```

Example : *inh3.cpp*

```
#include<iostream.h>  
#include<conio.h>  
class basecls  
{  
    private:  
        int n;  
    public:  
        void accept()  
        {  
            cin>>n;  
        }  
        void disp()  
        {  
            cout<<n;  
        }  
};  
class dercls:basecls  
{  
    public:  
        void accept()  
        {  
            basecls::accept();  
        }  
        void disp()  
        {  
            basecls::disp();  
        }  
};  
void main()  
{
```

```

    dercls d;
    clrscr();
    cout<<"Enter data in derived class : ";
    d.accept();
    cout<<"Data in the derived class    : ";
    d.disp();
    getch();
}

```

Data member overriding :

Example : inhr3.cpp

```

#include<iostream.h>
#include<conio.h>
class base
{
    protected:
        int n;
};
class der:base
{
    private:
        int n;
    public:
        void accept()
        {
            cout<<"Enter data in base class : ";
            cin>>base::n;
            cout<<"Enter data in derived class : ";
            cin>>n;
        }
        void disp()
        {
            cout<<"Data in base class : "<<base::n<<endl;
            cout<<"Data in derived class : "<<n;
        }
};
void main()
{

```

```
clrscr();  
der d;  
d.accept();  
d.disp();  
getch();  
}
```

Invoking base class constructors from derived class :

When both base and derived classes contain constructor, base class constructors are executed first and then the derived class constructors are executed.

In case of non parameterized constructors, the base class constructors are automatically executed at the time of declaration of derived class object. Because the base class object is internally created in derived class.

But in case of parameterized constructors, the constructors in derived class receives the entire list of values as its arguments and passes them on to the base class constructor.

syntax:

```
derv_cl( arg_list) : base_cl( parameter_list)  
{  
    statements;  
}
```

Example : (inh4.cpp)

Invoking base class constructor from derived class (non parameterized constructor)

```
#include<iostream.h>  
#include<conio.h>  
class basecls  
{
```

```

private:
    int n;
public:
    basecls()
    {
        n=100;
    }
    void disp()
    {
        cout<<n<<endl;
    }
};
class dercls:basecls
{
private:
    int n;
public:
    dercls()
    {
        n=200;
    }
    void dercls::disp()
    {
        cout<<"Data in Base class    : ";
        basecls::disp();
        cout<<"Data in Derived class : "<<n;
    }
};
void main()
{
    dercls d;
    clrscr();
    d.disp();
    getch();
}

```

Example : (inh5.cpp)

Invoking base class constructor from derived class (parameterized constructor)

```
#include<iostream.h>
#include<conio.h>
class basecls
{
    private:
        int n;
    public:
        basecls(int x)
        {
            n=x;
        }
        void disp()
        {
            cout<<n<<endl;
        }
};
class dercls:basecls
{
    private:
        int n;
    public:
        dercls(int x,int y):basecls(x)
        {
            n=y;
        }
        void dercls::disp()
        {
            cout<<"Data in Base class    : ";
            basecls::disp();
            cout<<"Data in Derived class : "<<n;
        }
};
void main()
{
    dercls d(100,200);
    clrscr();
    d.disp();
    getch();
}
```

```
}
```

Multiple inheritance :

One derived class with several base classes is called as multiple inheritance.

Syntax :

```
class derv_cl : [visibility mode] base_cl -1 ..., [visibility mode] base_cl-n
{
    Statements ;
};
```

..

.....

Example : (inh6.cpp)

```
#include<iostream.h>
#include<conio.h>
class base1
{
```

```

    protected:
        int x;
};
class base2
{
    protected:
        int y;
};
class der:base1,base2
{
    public:
        void accept()
        {
            cout<<"Enter data in base class 1 : ";
            cin>>x;
            cout<<"Enter data in base class 2 : ";
            cin>>y;
        }
        void disp()
        {
            cout<<"Given data in base class 1 : "<<x<<endl;
            cout<<"Given data in base class 2 : "<<y;
        }
};
void main()
{
    clrscr();
    der d;
    d.accept();
    d.disp();
    getch();
}

```

Example : inhr6.cpp

```

#include<iostream.h>
#include<conio.h>
class base1
{
    protected:

```

```

    int n;
public:
    void accept()
    {
        cin>>n;
    }
    void disp()
    {
        cout<<n<<endl;
    }
};
class base2
{
protected:
    int n;
public:
    void accept()
    {
        cin>>n;
    }
    void disp()
    {
        cout<<n<<endl;
    }
};
class der:base1,base2
{
public:
    void accept()
    {
        cout<<"Enter data in base class 1 : ";
        base1::accept();
        cout<<"Enter data in base class 2 : ";
        base2::accept();
    }
    void disp()
    {
        cout<<"Data in base class 1 : ";
        base1::disp();
        cout<<"Data in base class 2 : ";
    }
};

```

```

        base2::disp();
    }
};
void main()
{
    clrscr();
    der d;
    d.accept();
    d.disp();
    getch();
}

```

Example : (inh7.cpp)

```

#include<iomanip.h>
#include<conio.h>
#include<string.h>
class stu
{
    private:
        int sno;
        char sname[20];
    public:
        void accept();
        void disp();
};
void stu::accept()
{
    cout<<"Enter Student Number : ";
    cin>>sno;
    cout<<"Enter Student Name   : ";
    cin.ignore();
    cin.getline(sname,20);
}
void stu::disp()
{
    cout<<"Student Number : "<<sno<<endl;
    cout<<"Student Name   : "<<sname<<endl;
}
class res

```

```

{
    protected:
        int c,cpp,java,tot;
        float avg;
        char res[10],div[15];
        void accept();
        void cal();
        void disp();
};
void res::accept()
{
    cout<<"Enter marks in C      : ";
    cin>>c;
    cout<<"Enter marks in CPP    : ";
    cin>>cpp;
    cout<<"Enter marks in Java   : ";
    cin>>java;
    cout<<endl;
}
void res::cal()
{
    tot=c+cpp+java;
    avg=(float)tot/3;
    if(c>=50 && cpp>=50 && java>=50)
    {
        strcpy(res,"PASS");
        if(avg>=60)
            strcpy(div,"FIRST");
        else
            strcpy(div,"SECOND");
    }
    else
    {
        strcpy(res,"FAIL");
        strcpy(div,"NO DIVISION");
    }
}
void res::disp()
{
    cout<<"Marks in C      : "<<c<<endl;

```

```

    cout<<"Marks in CPP    : "<<cpp<<endl;
    cout<<"Marks in Java  : "<<java<<endl;
    cout<<"Total Marks    : "<<tot<<endl;
    cout<<"Average Marks   : "<<avg<<endl;
    cout<<"Result          : "<<res<<endl;
    cout<<"Division        : "<<div<<endl<<endl;
}
class student:stu,res
{
    public:
        void accept();
        void disp();
};
void student::accept()
{
    stu::accept();
    res::accept();
    res::cal();
}
void student::disp()
{
    stu::disp();
    res::disp();
}
void main()
{
    student s[100];
    int n;
    cout<<"Enter Number of Records : ";
    cin>>n;
    cout<<endl;
    for(int i=0;i<n;i++)
    {
        cout<<"Enter Student Record : "<<i+1<<endl;
        cout<<"-----"<<endl;
        s[i].accept();
    }
    clrscr();
    for(i=0;i<n;i++)
    {

```

```

        cout<<"Student Record : "<<i+1<<endl;
        cout<<"-----"<<endl;
        s[i].disp();
    }
}

```

Hierarchical inheritance :

One class may be inherited by more than one class. This process is called as hierarchical inheritance. i.e. one base class with several derived classes.

.....

Example : (inh8.cpp)

```

#include<iostream.h>
#include<conio.h>

```



```

class basecls
{
    protected:
        int x;
};
class der1:basecls
{
    private:
        int y;
    public:
        void accept()
        {
            cout<<"Enter data in the Base Class  : ";
            cin>>x;
            cout<<"Enter data in Derived Class 1 : ";
            cin>>y;
        }
        void disp()
        {
            cout<<"Data in the Base Class  : "<<x<<endl;
            cout<<"Data in Derived Class 1 : "<<y<<endl;
        }
};
class der2:basecls
{
    private:
        int z;
    public:
        void accept()
        {
            cout<<"Enter data in the Base Class  : ";
            cin>>x;
            cout<<"Enter data in Derived Class 2 : ";
            cin>>z;
        }
        void disp()
        {
            cout<<"Data in the Base Class  : "<<x<<endl;
            cout<<"Data in Derived Class 2 : "<<z<<endl;
        }
};

```

```
};
void main()
{
    der1 d1;
    der2 d2;
    clrscr();
    d1.accept();
    d2.accept();
    d1.disp();
    d2.disp();
    getch();
}
```

Example : (inh9.cpp)

```
#include<iostream.h>
#include<conio.h>
#include<string.h>
#include<process.h>

class student
{
    protected:
        int sno,tot;
        char sname[20];
        float avg;
        char res[10],div[15];
};
class languages:student
{
    private:
        int c,cpp,java;
    public:
        void accept();
        void cal();
        void disp();

};
void languages::accept()
{
```

```

    cout<<endl<<endl<<"Enter Student Number : ";
    cin>>sno;
    cout<<"Enter Student Name   : ";
    cin.ignore();
    cin.getline(sname,20);
    cout<<"Enter Marks in C     : ";
    cin>>c;
    cout<<"Enter Marks in CPP   : ";
    cin>>cpp;
    cout<<"Enter Marks in Java  : ";
    cin>>java;
}
void languages::cal()
{
    tot=c+cpp+java;
    avg=(float)tot/3;
    if(c>=50 && cpp>=50 && java>=50)
    {
        strcpy(res,"PASS");
        if(avg>=60)
            strcpy(div,"FIRST");
        else
            strcpy(div,"SECOND");
    }
    else
    {
        strcpy(res,"FAIL");
        strcpy(div,"NO DIVISION");
    }
}
void languages::disp()
{
    cout<<"Student Number : "<<sno<<endl;
    cout<<"Student Name   : "<<sname<<endl;
    cout<<"Marks in C     : "<<c<<endl;
    cout<<"Marks in CPP   : "<<cpp<<endl;
    cout<<"Marks in Java  : "<<java<<endl;
    cout<<"Total Marks   : "<<tot<<endl;
    cout<<"Average Marks  : "<<avg<<endl;
    cout<<"Result         : "<<res<<endl;
}

```

```

    cout<<"Division      : "<<div<<endl<<endl;
}
class packages:student
{
    private:
        int foxpro,oracle,msoff;
    public:
        void accept();
        void cal();
        void disp();
};
void packages::accept()
{
    cout<<endl<<endl<<"Enter Student Number  : ";
    cin>>sno;
    cout<<"Enter Student Name      : ";
    cin.ignore();
    cin.getline(sname,20);
    cout<<"Enter Marks in Foxpro : ";
    cin>>foxpro;
    cout<<"Enter Marks in Oracle : ";
    cin>>oracle;
    cout<<"Enter Marks in MS Off : ";
    cin>>msoff;
}
void packages::cal()
{
    tot=foxpro+oracle+msoff;
    avg=(float)tot/3;
    if(foxpro>=50 && oracle>=50 && msoff>=50)
    {
        strcpy(res,"PASS");
        if(avg>=60)
            strcpy(div,"FIRST");
        else
            strcpy(div,"SECOND");
    }
    else
    {
        strcpy(res,"FAIL");
    }
}

```

```

        strcpy(div,"NO DIVISION");
    }
}
void packages::disp()
{
    cout<<"Student Number    : "<<sno<<endl;
    cout<<"Student Name      : "<<sname<<endl;
    cout<<"Marks in Foxpro    : "<<foxpro<<endl;
    cout<<"Marks in Oracle     : "<<oracle<<endl;
    cout<<"Marks in MSOffice : "<<msoff<<endl;
    cout<<"Total Marks        : "<<tot<<endl;
    cout<<"Average Marks      : "<<avg<<endl;
    cout<<"Result              : "<<res<<endl;
    cout<<"Division            : "<<div<<endl<<endl;
}
void main()
{
    languages l;
    packages p;
    char ch;
    int n;
    clrscr();
    cout<<"1.Languages"<<endl<<endl;
    cout<<"2.Packages"<<endl<<endl;
    cout<<"-----"<<endl<<endl;
    do
    {
        cout<<"Enter category (1 or 2) : ";
        cin>>n;
        switch(n)
        {
            case 1:
            {
                l.accept();
                l.cal();
                clrscr();
                l.disp();
                break;
            }
            case 2:

```

```

    {
        p.accept();
        p.cal();
        clrscr();
        p.disp();
        break;
    }
    default :
    {
        cout<<"Invalid category"<<endl;
        break;
    }
}
cout<<"Do u want to add another record (yes or no) : ";
cin>>ch;
clrscr();
}
while(ch=='y' || ch=='Y');
exit(0);
getch();
}

```

Multi level inheritance :

The mechanism of deriving a class from another derived class is called as multi level inheritance.

Example : (*inh10.cpp*)

```

#include<iostream.h>
#include<conio.h>
class base
{
    public:
        int n;
};
class der1:public base
{
    protected:
        int n;
};
class der2:der1
{
    private:
        int n;
    public:
        void accept();
        void disp();
};
void der2::accept()
{
    cout<<"Enter data in the Base class : ";
    cin>>base::n;
    cout<<"Enter data in Derived class1 : ";
    cin>>der1::n;
    cout<<"Enter data in Derived class2 : ";
    cin>>der2::n;
}
void der2::disp()
{
    cout<<"Given data in the Base class : "<<base::n<<endl;
    cout<<"Given data in Derived class1 : "<<der1::n<<endl;
    cout<<"Given data in Derived class2 : "<<der2::n<<endl;
}
void main()
{
    der2 d;
    clrscr();

```

```

    d.accept();
    d.disp();
    getch();
}

```

Example : (inh11.cpp)

```

#include<iomanip.h>
#include<conio.h>
#include<string.h>
class student
{
    private:
        int sno;
        char sname[20];
    public:
        void accept();
        void disp();
};
void student::accept()
{
    cout<<"Enter Student Number : ";
    cin>>sno;
    cout<<"Enter Student Name   : ";
    cin.ignore();
    cin.getline(sname,20);
}
void student::disp()
{
    cout<<"Student Number : "<<sno<<endl;
    cout<<"Student Name   : "<<sname<<endl;
}
class marks:public student
{
    protected:
        int c,cpp,java;
        void accept();
        void disp();
};
void marks::accept()

```



```

{
    cout<<"Enter marks in C      : ";
    cin>>c;
    cout<<"Enter marks in CPP   : ";
    cin>>cpp;
    cout<<"Enter marks in Java  : ";
    cin>>java;
    cout<<endl;
}
void marks::disp()
{
    cout<<"Marks in C      : "<<c<<endl;
    cout<<"Marks in CPP   : "<<cpp<<endl;
    cout<<"Marks in Java  : "<<java<<endl;
};
class result:marks
{
    private:
        int tot;
        float avg;
        char res[10],div[15];
    public:
        void accept();
        void cal();
        void disp();
};
void result::accept()
{
    student::accept();
    marks::accept();
};
void result::cal()
{
    tot=c+cpp+java;
    avg=(float)tot/3;
    if(c>=50 && cpp>=50 && java>=50)
    {
        strcpy(res,"PASS");
        if(avg>=60)
            strcpy(div,"FIRST");
    }
}

```

```

        else
            strcpy(div,"SECOND");
    }
    else
    {
        strcpy(res,"FAIL");
        strcpy(div,"NO DIVISION");
    }
}
void result::disp()
{
    student::disp();
    marks::disp();
    cout<<"Total Marks    : "<<tot<<endl;
    cout<<"Average Marks  : "<<avg<<endl;
    cout<<"Result          : "<<res<<endl;
    cout<<"Division        : "<<div<<endl<<endl;
}
void main()
{
    clrscr();
    result r[100];
    char ch;
    int id=0,i;
    do
    {
        clrscr();
        cout<<"Enter Student Record"<<id+1<<endl<<endl;
        r[id].accept();
        r[id].cal();
        id++;
        cout<<"Do u want to continue(y/n) : ";
        cin>>ch;
    }
    while(ch=='y' && id<100);
    clrscr();
    for(i=0;i<id;i++)
    {
        clrscr();
        cout<<"Student Record "<<i+1<<endl<<endl;
    }
}

```

```
    r[i].disp();  
    cout<<"Press any key to continue....."<<endl;  
    getch();  
}  
}
```

Hybrid Inheritance :

In some situations, we need to apply 2 or more types of inheritance to design a program. Combining all these types of inheritances leads to new inheritance relationship between various classes, this gives hybrid inheritance.

Example : (inh12.cpp)

```
#include<iostream.h>  
#include<conio.h>  
class base  
{  
    protected:  
        int n;  
};  
class der1:base  
{
```

```

private:
    int n;
public:
    void accept();
    void disp();
};
void der1::accept()
{
    cout<<"Enter data in the Base Class  : ";
    cin>>base::n;
    cout<<"Enter data in Derived Class 1 : ";
    cin>>n;
}
void der1::disp()
{
    cout<<endl<<"Given data in the Base class  : "<<base::n<<endl;
    cout<<"Given data in Derived Class 1 : "<<n<<endl;
}
class der2:base
{
private:
    int n;
public:
    void accept();
    void disp();
};
void der2::accept()
{
    cout<<"Enter data in the Base Class  : ";
    cin>>base::n;
    cout<<"Enter data in Derived Class 2 : ";
    cin>>n;
}
void der2::disp()
{
    cout<<"Given data in the Base class  : "<<base::n<<endl;
    cout<<"Given data in Derived Class 2 : "<<n<<endl;
}
class der3:der1,der2
{

```

```

    public:
        void accept();
        void disp();
};
void der3::accept()
{
    der1::accept();
    der2::accept();
}
void der3::disp()
{
    der1::disp();
    der2::disp();
}
void main()
{
    der3 d;
    clrscr();
    d.accept();
    d.disp();
    getch();
}

```

Virtual base class :

In the above diagram, all the 3 kinds of inheritance namely multi level, multiple, and hierarchical inheritance are involved. The Derv1 and Derv2 classes are publicly or protectedly inherited from Base class. So all the public and protected members of Base class are inherited into Derv3 twice (via Derv1 and Derv2), It

means Derv3 would have duplicated sets of numbers inherited from Base. This introduces ambiguity and should be avoided.

When a class is made a virtual base class, C++ takes necessary care to see that only one copy of the class is inherited regardless of how many inheritance paths exists between virtual base class and derived classes.

Example : (inh13.cpp)

```
#include<iostream.h>
#include<conio.h>
class base
{
    protected:
        int n;
};
class der1:virtual protected base
{
    protected:
        int a;
};
class der2:virtual protected base
{
    protected:
        int b;
};
class der3:der1,der2
{
    public:
        void accept();
};
```

```

        void disp();
};
void der3::accept()
{
    cout<<"Enter data in the Base class  : ";
    cin>>n;
    cout<<"Enter data in Derived class 1 : ";
    cin>>a;
    cout<<"Enter data in Derived class 2 : ";
    cin>>b;
}
void der3::disp()
{
    cout<<endl<<"Given data in the Base class  : "<<n<<endl;
    cout<<"Given data in Derived class 1 : "<<a<<endl;
    cout<<"Given data in Derived class 2 : "<<b;
}
void main()
{
    der3 d;
    clrscr();
    d.accept();
    d.disp();
    getch();
}

```

Example : inh14.cpp

To display student details of science, arts and engineering and calculate total marks and their result and division

```
#include<iostream.h>
#include<conio.h>
#include<iomanip.h>
#include<string.h>
class student
{
    protected:
        int sno;
        char sname[20];
};
class language
{
    protected:
        int lan1,lan2;
};
class science:virtual protected student,virtual protected language
{
};
class arts:virtual protected student,virtual protected language
{
};
class engineering:virtual protected student
{
```



```

};
class bsc_mpc:virtual protected science
{
    protected:
        int maths,physics;
};
class bsc_cbz:virtual protected science
{
    protected:
        int botony,zoology;
};
class ba:virtual protected arts
{
    protected:
        int history,politics;
};
class bcom:virtual protected arts
{
    protected:
        int accounts,economics;
};
class computers:virtual protected engineering
{
    protected:
        int java,unix;
};
class electronics:virtual protected engineering
{
    protected:
        int gates,lc;
};
class result:bsc_mpc,bsc_cbz,ba,bcom,computers,electronics
{
    private:
        int tot;
        float avg;
        char res[10],div[15];
    public:
        char * getdata(char *);
        void store_res(int,int,int,int);
};

```

```

    void store_res(int,int);
    void cal_res(int,int);
    void disp_res(char *,char *);
};
char * result::getdata(char *st)
{
    char *s;
    s=new char[10];
    cout<<"Enter Student Number : ";
    cin>>sno;
    cout<<"Enter Student Name    : ";
    cin.ignore();
    cin.getline(sname,20);
    if(stricmp(st,"arts")==0 || stricmp(st,"science")==0)
    {
        cout<<"Enter marks in First language  : ";
        cin>>lan1;
        cout<<"Enter marks in Second language : ";
        cin>>lan2;
        if(stricmp(st,"science")==0)
        {
            lb1:
            cout<<"Enter Bsc_MPC/Bsc_CBZ : ";
            cin>>s;
            if(stricmp(s,"Bsc_MPC")==0)
            {
                cout<<"Enter marks in Maths : ";
                cin>>maths;
                cout<<"Enter marks in Physics : ";
                cin>>physics;
            }
            else if(stricmp(s,"Bsc_CBZ")==0)
            {
                cout<<"Enter marks in Botony : ";
                cin>>botony;
                cout<<"Enter marks in Zoology : ";
                cin>>zoology;
            }
            else
            {

```

```

        cout<<"Invalid Option"<<endl;
        goto lb1;
    }
}
else
{
    lb2:
    cout<<"Enter BA/BCom : ";
    cin>>s;
    if(stricmp(s,"BA")==0)
    {
        cout<<"Enter marks in History : ";
        cin>>history;
        cout<<"Enter marks in Politics : ";
        cin>>politics;
    }
    else if(stricmp(s,"BCom")==0)
    {
        cout<<"Enter marks in Accounts : ";
        cin>>accounts;
        cout<<"Enter marks in Economics : ";
        cin>>economics;
    }
    else
    {
        cout<<"Invalid Option"<<endl;
        goto lb2;
    }
}
}
else
{
    lb3:
    cout<<"Enter Comp/Elec : ";
    cin>>s;
    if(stricmp(s,"Comp")==0)
    {
        cout<<"Enter marks in Java : ";
        cin>>java;
        cout<<"Enter marks in Unix : ";

```

```

        cin>>unix;
    }
    else if(stricmp(s,"Elec")==0)
    {
        cout<<"Enter marks in Gates : ";
        cin>>gates;
        cout<<"Enter marks in Linear Circuits : ";
        cin>>lc;
    }
    else
    {
        cout<<"Invalid Option"<<endl;
        goto lb3;
    }
}
return s;
}
void result::store_res(int l1,int l2,int m1,int m2)
{
    if(l1>=35 && l2>=35 && m1>=35 && m2>=35)
        strcpy(res,"PASS");
    else
        strcpy(res,"FAIL");
}
void result::store_res(int m1,int m2)
{
    if(m1>=35 && m2>=35)
        strcpy(res,"PASS");
    else
        strcpy(res,"FAIL");
}
void result::cal_res(int m1,int m2)
{
    tot=m1+m2;
    avg=(float)tot/2;
    if(stricmp(res,"PASS")==0)
    {
        if(avg>=60)
            strcpy(div,"FIRST");
        else if(avg>=50)

```

```

        strcpy(div,"SECOND");
    else
        strcpy(div,"THIRD");
    }
else
{
    strcpy(res,"FAIL");
    strcpy(res,"NO DIVISION");
}
}
void result::disp_res(char *st,char *s)
{
    cout<<"Student Number : "<<sno<<endl;
    cout<<"Student Name   : "<<sname<<endl;
    if(stricmp(st,"science")==0 || stricmp(st,"arts")==0)
    {
        cout<<"Marks in First Language  : "<<lan1<<endl;
        cout<<"Marks in Second Language : "<<lan2<<endl;
    }
    if(stricmp(st,"science")==0)
    {
        if(stricmp(s,"Bsc_MPC")==0)
        {
            store_res(lan1,lan2,maths,physics);
            cal_res(maths,physics);
            cout<<"Marks in Maths : "<<maths<<endl;
            cout<<"Marks in Physics : "<<physics<<endl;
        }
        else
        {
            store_res(lan1,lan2,botony,zoology);
            cal_res(botony,zoology);
            cout<<"Marks in Botony : "<<botony<<endl;
            cout<<"Marks in Zoology : "<<zoology<<endl;
        }
    }
    else if(stricmp(st,"arts")==0)
    {
        if(stricmp(s,"BA")==0)
        {

```

```

        store_res(lan1,lan2,history,politics);
        cal_res(history,politics);
        cout<<"Marks in History : "<<history<<endl;
        cout<<"Marks in Politics : "<<politics<<endl;
    }
    else
    {
        store_res(lan1,lan2,accounts,economics);
        cal_res(accounts,economics);
        cout<<"Marks in Accounts : "<<accounts<<endl;
        cout<<"Marks in Economics : "<<economics<<endl;
    }
}
else
{
    if(stricmp(s,"comp")==0)
    {
        store_res(java,unix);
        cal_res(java,unix);
        cout<<"Marks in Java : "<<java<<endl;
        cout<<"Marks in Unix : "<<unix<<endl;
    }
    else
    {
        store_res(gates,lc);
        cal_res(gates,lc);
        cout<<"Marks in Gates : "<<gates<<endl;
        cout<<"Marks in LC : "<<lc<<endl;
    }
}
cout<<"Total Marks : "<<tot<<endl;
cout<<"Average Marks : "<<avg<<endl;
cout<<"Result : "<<res<<endl;
cout<<"Division : "<<div<<endl;
}
void main()
{
    char st[5],s[10],ch;
    result r;
    cout<<setiosflags(ios::showpoint);

```

```

cout<<setprecision(2);
do
{
    clrscr();
    cout<<"Enter sci/arts/engg : ";
    cin>>st;
    if(stricmp(st,"sci") || stricmp(st,"arts") || stricmp(st,"engg")==0)
    {
        strcpy(s,r.getdata(st));
        clrscr();
        r.disp_res(st,s);
    }
    else
        cout<<"Invalid option"<<endl;
    cout<<endl<<"Do u want to continue (y/n) : ";
    cin>>ch;
}
while(ch=='y');
}

```

Polymorphism and Virtual functions

Polymorphism is nothing but the ability to access different implementation of the function using same name. There are 2 levels at which the polymorphism operates.

- 1) Compile time polymorphism (Static or Early binding)
- 2) Run time polymorphism (Dynamic or Late binding)

We have already seen how the concept of polymorphism is implemented using the overloaded functions and operators. The overloaded member functions are selected for invoking by matching arguments. This information is known to the compiler at the compile time and therefore the compiler is able to select

the appropriate function for a particular call at the compile time itself. This is called early binding or static binding and also known as compile time polymorphism. Early binding means that an object is bounded to its function call at compile time.

If the appropriate member function could be selected while the program is running, It is known as runtime polymorphism. C++ supports a mechanism known as virtual functions to achieve runtime polymorphism. At runtime when it is known what class objects are under consideration, the appropriate version of the function is invoked. Since the function is linked with a particular class, much later after the compilation. This is called late binding, because the selection of appropriate function is done dynamically at run time.

Example : poly1.cpp

```
#include<iostream.h>
#include<conio.h>
class base
{
    public:
        void disp()
        {
            cout<<"This is Base Class"<<endl;
        }
};
class der1:public base
{
    public:
        void disp()
        {
            cout<<"This is Derived Class-1";
```



```

    }
};
class der2:public base
{
    public:
        void disp()
        {
            cout<<"This is Derived Class-2";
        }
};

void main()
{
    base b,*p;
    der1 d1;
    der2 d2;
    clrscr();
    p=&b;
    p->disp();
    p=&d1;
    p->disp();
    p=&d2;
    p->disp();
    getch();
}

```

Output

This is Base class
This is Base class
This is Base class

Above program displays 3 same messages (This is Base class). Because it is compile time polymorphism. The base class methods are strongly bounded to an object at compile time.

Virtual functions :

When we use the same function name (prototypes are same) in both base and

derived classes, the function in base class is declared as virtual using the keyword **virtual** preceded its normal declaration. When a function is made virtual, C++ determines which function to use at run time based on the type of objects pointed by the base pointer. Thus by making the base pointer to point to different objects we can execute different versions of the virtual functions.

Example : poly2.cpp

```
#include<iostream.h>
#include<conio.h>
class base
{
    public:
        virtual void disp()
        {
            cout<<"This is Base Class"<<endl;
        }
};
class der1:public base
{
    public:
        void disp()
        {
            cout<<"This is Derived Class-1"<<endl;
        }
};
class der2:public base
{
    public:
        void disp()
        {
            cout<<"This is Derived Class-2";
        }
};
void main()
{
    base b,*p;
    der1 d1;
    der2 d2;
    clrscr();
```

```
p=&b;  
p->disp();  
p=&d1;  
p->disp();  
p=&d2;  
p->disp();  
getch();  
}
```

Output

This is Base class
This is Derived class-1
This is Derived class-2

Here we have declared the base class method as static and hence the objects are bounded to their respective methods at their compile time and hence the pointer object execute the 3 disp() methods.

Note :

The visibility mode of derived classes must be public

Rules for Virtual functions :

When virtual functions are created for implementing late binding, we should observe some basic rules, that satisfy the compile requirements.

1. The virtual functions must be members of same class.
2. They cannot be static members.
3. They are accessed by using object pointers.
4. Virtual function in a base class must be defined even though it may not be used.
5. The prototypes of the base class version of a virtual function and all the derived class version must be identical. If two functions with the same name have different prototypes, C++ considered them as overloaded functions and the virtual function mechanism is ignored.
6. We cannot have virtual constructors but we can have virtual destructors.

Invoking non version of a virtual function (Using Type casting) :

If we want to invoke non version of a virtual function in derived class from base pointer we can apply type casting.

Example : virt1.cpp

```
#include<iostream.h>
#include<conio.h>
class base
{
    public:
        virtual void disp()
        {
            cout<<"This is base class disp"<<endl;
        }
};
class der:public base
{
    public:
        void disp()
        {
            cout<<"This is derived class disp"<<endl;
        }
        void show()
        {
            cout<<"This is derived class show";
        }
};
void main()
{
    base b,*p;
    der d;
    clrscr();
    p=&b;
    p->disp();
```

```

p=&d;
p->disp();
((der *)p)->show();
getch();
}

```

type cast to derived class pointer

Virtual destructors :

Example : *vir_dest.cpp*

```

#include<iostream.h>
#include<conio.h>
class base
{
public:
    base()
    {
        cout<<"Base class Constructor is invoked"<<endl;
    }
    virtual ~base()
    {
        cout<<"Base class Destructor is invoked"<<endl;
    }
};
class der:public base
{
public:
    der()
    {
        cout<<"Derived class Constructor is invoked"<<endl;
    }
    ~der()
    {
        cout<<"Derived class  Destructor is invoked"<<endl;
    }
};

```

```

void main()
{
    base *p;
    clrscr();
    p=new der;
    delete p;
    getch();
}

```

Without giving the keyword **virtual** in base class destructor, then the derived class destructor will not worked.

Pure virtual functions :

Declaring a virtual function without having definition is called as pure virtual function and it is initialized to zero.

Syntax :

```
virtual return_type func_name ([arg_list]) = 0;
```

Abstract class :

It is a class which cannot be used to create objects of the class. Because it contains at least one pure virtual function, it can always used as a base class.

Example : pur_vir1.cpp

```

#include<iostream.h>
#include<conio.h>
class base
{
    public:
        virtual void disp()=0;
};
class der1:public base
{
    public:
        void disp()
        {

```

```

        cout<<"This is derived class-1 disp"<<endl;
    }
};

class der2:public base
{
    public:
        void disp()
        {
            cout<<"This is derived class-2 disp"<<endl;
        }
};

void main()
{
    base *p;
    clrscr();
    p=new der1;
    p->disp();
    p=new der2;
    p->disp();
    getch();
}

```

Example : pur_vir2.cpp

```

#include<iostream.h>
#include<conio.h>
class ari
{
    public:
        virtual void compute(int,int)=0;
};

class add:public ari
{
    public:
        void compute(int x,int y)
        {
            cout<<"Addition = "<<x+y<<endl;
        }
};

```

```

class sub:public ari
{
    public:
        void compute(int x,int y)
        {
            cout<<"Subtraction = "<<x-y<<endl;
        }
};
void main()
{
    ari *p;
    int a,b;
    clrscr();
    cout<<"Enter 2 values : ";
    cin>>a>>b;
    p=new add;
    p->compute(a,b);
    p=new sub;
    p->compute(a,b);
    getch();
}

```

itoa :

It is a function which is included in the header file <stdlib.h>
It converts an integer to a string.

Syntax :

```
char * itoa(int value, char * string, int radix);
```

Example : pur_vir3.cpp

```

#include<iostream.h>
#include<conio.h>
#include<string.h>
#include<stdlib.h>
class vehicle

```



```

{
    public:
        virtual void accept()=0;
        virtual void disp()=0;
        virtual void inc()=0;
};
class twowheeler:public vehicle
{
    private:
        char name[20],vname[20],cname[20],regno[20],st[20];
        int n;
    public:
        void accept();
        void disp();
        void inc();
        twowheeler()
        {
            n=1000;
        }
};
void twowheeler::accept()
{
    cout<<"Enter Vehicle holder name : ";
    cin.ignore();
    cin.getline(name,20);
    cout<<"Enter Vehicle name : ";
    cin.getline(vname,20);
    cout<<"Enter Vehicle company name : ";
    cin.getline(cname,20);
    strcpy(regno,"AP16 TW");
    itoa(n,st,10);
    strcat(regno,st);
}
void twowheeler::disp()
{
    cout<<"Vehicle holder name  : "<<name<<endl;
    cout<<"Vehicle name          : "<<vname<<endl;
    cout<<"Vehicle company name : "<<cname<<endl;
    cout<<"Vehicle Reg numbr    : "<<regno<<endl;
}

```

```

void twowheeler::inc()
{
    n++;
}
class fourwheeler:public vehicle
{
    private:
        char name[20],vname[20],cname[20],regno[20],st[20];
        int n;
    public:
        void accept();
        void disp();
        void inc();
        fourwheeler()
        {
            n=1000;
        }
};
void fourwheeler::accept()
{
    cout<<"Enter Vehicle holder name : ";
    cin.ignore();
    cin.getline(name,20);
    cout<<"Enter Vehicle name : ";
    cin.getline(vname,20);
    cout<<"Enter Vehicle company name : ";
    cin.getline(cname,20);
    strcpy(regno,"AP16 FW");
    itoa(n,st,10);
    strcat(regno,st);
}
void fourwheeler::disp()
{
    cout<<"Vehicle holder name  : "<<name<<endl;
    cout<<"Vehicle name          : "<<vname<<endl;
    cout<<"Vehicle company name : "<<cname<<endl;
    cout<<"Vehicle Reg numbr    : "<<regno<<endl;
}
void fourwheeler::inc()
{

```

```

    n++;
}

void main()
{
    vehicle *v;
    char ch,st[5];
    twowheeler tw;
    fourwheeler fw;
    do
    {
        clrscr();
        cout<<"Enter two wheeler or four wheeler(tw/fw) : ";
        cin>>st;
        if(stricmp(st,"tw")==0)
        {
            v=&tw;
            v->accept();
            clrscr();
            v->disp();
            v->inc();
        }
        else if(stricmp(st,"fw")==0)
        {
            v=&fw;
            v->accept();
            clrscr();
            v->disp();
            v->inc();
        }
        else
            cout<<"Invalid Option";
        cout<<"Do you want to continue (y/n) : ";
        cin>>ch;
    }
    while(ch!='n');
}

```

We have used itoa() function in accept() method. This function is included in the header file **<stdlib.h>**, which converts an integer to a string and the third argument

of that function i.e. 10 indicates the base of integer is binary or octal or decimal or hexa decimal. In this case it is decimal.

Templates

It is a new concept which enable us to define generic classes, generic functions and thus provides support for generic programming. Generic programming is an approach where generic types are used as parameters, so that they work for a variety of suitable data types and data structures.

Template is a mechanism that make it possible to use one function or class to handle many different data types. By using templates we can design a single class or function that operates on data of many data types. When used with function they are known as "function templates or generic functions", where as when used with classes they are called as "class templates or generic classes".

Function templates or Generic functions

Syntax :

```
template < class t1, class t2, ....., class tn >
returntype funcname ([arg_list])
{
    statements;
```

```
}
```

Example : templt1.cpp

```
#include<iostream.h>
#include<conio.h>
template <class type>
void func(type var)
{
    cout<<"var = "<<var<<endl<<endl;
}
void main()
{
    clrscr();
    cout<<"Calling-1"<<endl;
    func(1000);
    cout<<"Calling-2"<<endl;
    func(14.17);
    cout<<"Calling-3"<<endl;
    func('R');
    cout<<"Calling-4"<<endl;
    func("RAJI");
    getch();
}
```

Example : templt2.cpp

To swap 2 data types using templates

```
#include<iostream.h>
#include<conio.h>
template <class type>
void swap(type &var1,type &var2)
{
    type t;
```

```

    t=var1;
    var1=var2;
    var2=t;
}
void main()
{
    int a,b;
    char c,d;
    clrscr();
    cout<<"Enter 2 numbers    : ";
    cin>>a>>b;
    cout<<"Before swaping"<<endl;
    cout<<"a = "<<a<<endl;
    cout<<"b = "<<b<<endl;
    swap(a,b);
    cout<<"After swaping"<<endl;
    cout<<"a = "<<a<<endl;
    cout<<"b = "<<b<<endl;
    cout<<"Enter 2 characters : ";
    cin>>c>>d;
    cout<<"Before swaping"<<endl;
    cout<<"c = "<<c<<endl;
    cout<<"d = "<<d<<endl;
    swap(c,d);
    cout<<"After swaping"<<endl;
    cout<<"c = "<<c<<endl;
    cout<<"d = "<<d<<endl;
    getch();
}

```

Example : templt3.cpp

To sort 2 data types using bubble sort

```

#include<iostream.h>
#include<conio.h>
#include<iomanip.h>
template <class type>
void bsort(type a[],type n)
{

```

```

int i,j;
type t;
for(i=0;i<n-1;i++)
{
    for(j=0;j<n-i-1;j++)
    {
        if(a[j]>a[j+1])
        {
            t=a[j];
            a[j]=a[j+1];
            a[j+1]=t;
        }
    }
}
}
void main()
{
    clrscr();
    int a[20],n,i;
    cout<<"Enter number of Elements : ";
    cin>>n;
    cout<<"Enter Elements : "<<endl;
    for(i=0;i<n;i++)
    {
        cin>>a[i];
    }
    cout<<"Elements before Sorting: "<<endl;
    for(i=0;i<n;i++)
    {
        cout<<setw(5)<<a[i];
    }
    bsort(a,n);
    cout<<endl<<"Elements after Sorting: "<<endl;
    for(i=0;i<n;i++)
    {
        cout<<setw(5)<<a[i];
    }
    getch();
}

```

Overloading of template functions :

A template function may be overloaded either by template function or ordinary function of its name. In such cases the over loading resolution is accomplish as follows :

- 1) Call an ordinary function that has an exact match
- 2) Call a template function that could be created with an exact match.

Example : *templt4.cpp*

```
#include<iostream.h>
#include<conio.h>
template <class type>
void func(type var)
{
    cout<<"Template Function : 1"<<endl;
    cout<<"var = "<<var<<endl<<endl;
}
template <class type1,class type2>
void func(type1 var1,type2 var2)
{
    cout<<"Template Function : 2"<<endl;
    cout<<"var1 = "<<var1<<endl<<endl;
    cout<<"var2 = "<<var2<<endl<<endl;
}
void func(int var)
{
    cout<<"Normal Function"<<endl;
    cout<<"var = "<<var<<endl<<endl;
}
void main()
{
    clrscr();
    func("RAJI");
```



```
func('R');  
func(100);  
func(14.17);  
func("raji","ram");  
func(1814,1714);  
getch();  
}
```

Output

Template Function : 1
var = RAJI

Template Function : 2
var = R

Normal Function
var = 100

Template Function : 1
var = 14.17

Template Function : 2
var1 = raji
var2 = ram

Template Function : 2
var1 = 1814
var2 = 1714

Class templates (or) Generic classes :

Syntax :

```

template < class t1, class t2, ....., class tn >
class class_name
{
    Class definition;
};

```

Declaration of an object of a template class :

```

class_name    <datatype_1, datatype_2,.....,datatype_n > obj_name ;

```

Example : templt5.cpp

```

#include<iostream.h>
#include<conio.h>
template <class type>
class test
{
    private:
        type var;
    public:
        void accept()
        {
            cin>>var;
        }
        void disp()
        {
            cout<<var<<endl;
        }
};
void main()
{
    clrscr();
    test <int>t1;
    cout<<"Enter any integer : ";
}

```

```

t1.accept();
cout<<"Given integer    : ";
t1.disp();
test <float>t2;
cout<<"Enter any float : ";
t2.accept();
cout<<"Given float      : ";
t2.disp();
test <char>t3;
cout<<"Enter any character : ";
t3.accept();
cout<<"Given character    : ";
t3.disp();
test <char *>t4;
cout<<"Enter any string : ";
t4.accept();
cout<<"Given string      : ";
t4.disp();
getch();
}

```

Defining member function outside the template class :

```

template<class t1, class t2, ....., class tn>
class class_name
{
    body;
};

```

```

template<class t1, class t2, ....., class tn>
return_type  class_name  <t1,t2,....., tn> :: func_name([arg_list]);
{
    body;
}

```

Example : templt6.cpp

```

#include<iostream.h>
#include<conio.h>
template<class type>

```

```

class test
{
    private:
        type var;
    public:
        void accept();
        void disp();
};

template<class type>
void test<type>::accept()
{
    cin>>var;
}

template <class type>
void test<type>::disp()
{
    cout<<var<<endl;
}

void main()
{
    clrscr();
    test <int>t1;
    cout<<"Enter any integer : ";
    t1.accept();
    cout<<"Given integer    : ";
    t1.disp();
    test <float>t2;
    cout<<"Enter any float : ";
    t2.accept();
    cout<<"Given float      : ";
    t2.disp();
    test <char>t3;
    cout<<"Enter any character : ";
    t3.accept();
    cout<<"Given character    : ";
    t3.disp();
    test <char *>t4;
    cout<<"Enter any string : ";
    t4.accept();
    cout<<"Given string      : ";
}

```

```
t4.disp();
getch();
}
```

Non type template arguments :

We have seen a template can have multiple arguments. It is also possible to use non type template arguments.

Note :

Non type template argument must be “constant value”

Example : templt7.cpp

```
#include<iomanip.h>
#include<iostream.h>
#include<conio.h>
template <class type,int n>
class arr
{
    private:
        type a[n];
    public:
        void accept();
        void disp();
};
template <class type,int n>
void arr<type,n>::accept()
{
    int i;
    for(i=0;i<n;i++)
    {
        cin>>a[i];
    }
}
template <class type,int n>
void arr<type,n>::disp()
```

```

{
    int i;
    for(i=0;i<n;i++)
    {
        cout<<setw(5)<<a[i];
    }
}
void main()
{
    arr<int,5>obj;
    cout<<"Enter 5 elements : " ;
    obj.accept();
    cout<<"Given 5 elements : ";
    obj.disp();
}

```

Inheriting template classes :

Example : templt8.cpp

```

#include<iostream.h>
#include<conio.h>
template<class type>
class base
{
    private:
        type var;
    public:
        void accept();
        void disp();
};
template<class type>
void base<type>::accept()
{
    cin>>var;
}
template<class type>
void base<type>::disp()
{
    cout<<var<<endl;
}

```

```

}
template<class type>
class der:base<type>
{
};
void main()
{
    clrscr();
    der<int>d;
    cout<<"Enter any data : ";
    d.accept();
    cout<<"Given data      : ";
    d.disp();
    getch();
}

```

Overriding :

Example : templt9.cpp

```

#include<iostream.h>
#include<conio.h>
template<class type>
class base
{
    private:
        type var;
    public:
        void accept()
        {
            cin>>var;
        }
        void disp()
        {
            cout<<var<<endl;
        }
};
template<class type>
class der:public base<type>
{

```

```

public:
    void accept()
    {
        base<type>::accept();
    }
    void disp()
    {
        base<type>::disp();
    }
};
void main()
{
    clrscr();
    der<int>d;
    cout<<"Enter any data : ";
    d.accept();
    cout<<"Given data      : ";
    d.disp();
    test
    getch();
}

```

Namespaces :

We have been defining variables in different scopes in CPP programs such as classes, functions, blocks etc. ANSI C++ standard has added a new keyword ***namespace*** to define

a scope that could hold global identifiers.

Defining namespaces :

We can define our own name spaces in ur programs. The syntax for defining namespace is similar to syntax for defining a class.

The general form of namespace is :

```
namespace namespace_name
{
    Declaration of variables, functions,
    classes etc;
}
```

using :

It is a directive and is used to access the members of a name space.

Syntax : using namespace namespace_name;

Example : nm_sp1.cpp

```
#include<iostream.h>
#include<conio.h>
namespace nm
{
```

```

        int n=100;
    }

void main()
{
    using namespace nm;
    cout<<"Value of n : "<<n<<endl;
    getch();
}

```

Example : nm_sp2.cpp

```

#include<iostream.h>
#include<conio.h>
namespace nm1
{
    int n=100;
}
namespace nm2
{
    Int n=200;
}
void main()
{
    using namespace nm1;
    using namespace nm2;
    cout<<"Value of n in nm1 : "<<nm1::n<<endl;
}

```

```
        cout<<"Value of n in nm2 : "<<nm2::n<<endl;
        getch();
    }
```

Example : nm_sp3.cpp

```
#include<iostream.h>
#include<conio.h>
namespace nm1
{
    void disp()
    {
        cout<<"Name space - 1 "<<endl;
    }
}
namespace nm2
{
    void disp()
    {
        cout<<"Name space - 2 "<<endl;
    }
}

void main()
{
    using namespace nm1;
    using namespace nm2;
    nm1::disp();
    nm2::disp();
}
```

```
}
```

Example : nm_sp4.cpp

```
#include<iostream.h>
#include<conio.h>
namespace myns
{
    class test
    {
        private:
            int n;
        public:
            void accept()
            {
                cin>>n;
            }
            void disp()
            {
                cout<<n<<endl;
            }
    };
}
void main()
{
    using namespace myns;
    test t;
    cout<<"Enter any data : ";
    t.accept();
    cout<<"Given data : ";
    t.disp();
}
```

```
    getch();  
}
```

(Main usage of name space is if we declare any members inside a name space, we can not use those members any where like other variables, we should use those variables where we need with the help of **using** keyword)

Exception handling :

Exception is an error state. It occurs during the execution of an instruction in a program. For an exception occurs generally, the system may be in any one of the following states.

1. Unstable or hang.
2. System may went into an infinite loop.
3. Breaking the execution of an application by showing an error message.

But because of these states of a system, it is not possible to execute any application properly. So when an exception occurs it is possible to raise an exception by the system and when an exception is raised a set of instructions will be executed and these instructions are refereed as “Exception Handler”.

try :

It is used to write instructions which are having a chance to raise exceptions.

Syntax :

```
try
{
    Statements;
}
```

catch :

It is used to write an exception handler, which consists a set of instructions that are required to execute, in response to the error condition.

Syntax :

```
catch (datatype arg)
{
    Statements;
}
```

throw :

When exception that is desired to be handle is detected, it is thrown using the throw statement.

Syntax :

```
throw(parameter);
```

where parameter, is the argument of catch block.

Example : **excp1.cpp**

```

#include<iostream.h>
#include<conio.h>
void main()
{
    int a,b;
    float c;
    cout<<"Enter any two values : ";
    cin>>a>>b;
    try
    {
        if(b==0)
            throw(1);
        c=(float)a/b;
        cout<<"Division = "<<c<<endl;
    }
    catch(int )
    {
        cout<<"Division Error";
    }
}

```

Multiple catch statements :

Example : *excp2.cpp*

```

#include<iostream.h>

```

```

#include<conio.h>
void main()
{
    int n;
    char ch;
    float f;
    try
    {
        cout<<"Enter any integer : ";
        cin>>n;
        if(n==0)
            throw(n);
        cout<<"Enter any character : ";
        cin>>ch;
        if(ch<65 || (ch>90 && ch<97) || ch>122)
            throw(ch);
        cout<<"Enter any float : ";
        cin>>f;
        if(f==0)
            throw(f);

        cout<<"Given Integer      : "<<n<<endl;
        cout<<"Given Character    : "<<ch<<endl;
        cout<<"Given Float        : "<<f;
    }
    catch(int)
    {
        cout<<"Integer error"<<endl;
    }
}

```



```

catch(char)
{
    cout<<"Character error"<<endl;
}
catch(float)
{
    cout<<"Float error";
}
}

```

Multiple Exceptions and single Exception Handler (catch statement) :

Example : excp3.cpp

```

#include<iostream.h>
#include<conio.h>
void main()
{
    int n;
    char ch;
    float f;
    try
    {
        cout<<"Enter any integer : ";
        cin>>n;
        if(n==0)
            throw(n);
    }
}

```

```

    cout<<"Enter any character : ";
    cin>>ch;
    if(ch<65 || (ch>90 && ch<97) || ch>122)
        throw(ch);
    cout<<"Enter any float : ";
    cin>>f;
    if(f==0)
        throw(f);
    cout<<"Given Integer : "<<n<<endl;
    cout<<"Given Character : "<<ch<<endl;
    cout<<"Given Float : "<<f<<endl;
}
catch(...)
{
    cout<<"Error occurred"<<endl;
}
}

```

Rethrowing the exception :

Example : excp4.cpp

```

#include<iostream.h>
#include<conio.h>
float div(int x,int y)
{
    float d;
    try
    {

```

```

        if(y==0)
            throw(0);
        d=(float)x/y;
    }
    catch(int)
    {
        cout<<"Error in Function block"<<endl;
        throw;    // rethrowing the exception
    }
    return d;
}
void main()
{
    int a,b;
    float c;
    cout<<"Enter 2 numbers : ";
    cin>>a>>b;
    try
    {
        c=div(a,b);
        cout<<"Division = "<<c<<endl;
    }
    catch(...)
    {
        cout<<"Division Error"<<endl;
    }
}

```

If there is any error in the function block, it will execute catch block and here again it is rethrowing to the main block and again

it execute catch of main() block.

I/O operations in C++ (Files and streams):

C++ streams :

C++ uses the concept of streams and stream classes to implement its I/O operations with console and disk files.

A stream is a sequence of bytes, a stream is a general name given to the flow of data. Different streams are used to represent different kinds of data flow. The source stream that provides data to the program is called input stream and the destination stream that receives output from the program is called output stream.

C++ stream classes :

The I/O system of c++ contains a hierarchy of classes that are used to define various streams to deal with both the console and disk files. These classes are called stream classes.

Stream classes for console IO operations :

ios (general ios stream class) : It provides the basic support for formatted and unformatted I/O operations.

istream : It provides facilities for formatted and unformatted input operations.

ostream : It provides facilities for formatted and unformatted output operations.

iostream : It provides facilities for both input and output operations.

stream buffer : The ios class contains a pointer to a stream buffer class, which contains the actual memory buffers into which the data is read or write and the routines to handle its

data. It provides an interface to physical devices through buffer.

Unformatted IO operations :

- 1) overloaded operators insertion(<<), extraction(>>), cin and cout objects.
- 2) put and get functions
- 3) getline and write functions.

get : (istream :: get (member function))

Form 1 : int get();

It extracts a single character from input stream.

Form 2 : istream& get(char &ch);

It extracts a single character into the given character reference.

Form 3 : istream& get(char *s, int len , char ch= '\n');

It extracts specified number of character from input stream.

put : (ostream :: put (member function))

It inserts a single character into the output stream.

Syntax : ostream& put(char ch);

flush: (ostream :: flush (member function))

It flushes the output stream.

Syntax : ostream& flush();

getline: (istream :: getline(member function))

istream & getline(char *s,int n,char ch='\n');

write : (ostream :: write(member function))

It insets specified number of characters into the output stream.

Syntax : ostream& write(const char *, int n);

Example : **io1.cpp**

```
#include<iostream.h>
#include<conio.h>
void main()
{
    char ch;
    clrscr();
    cout<<"Enter any character : ";
```

```

    ch=cin.get();    // form : 1
// cin.get(ch);    form : 2
    cout<<"Given character is  : ";
    cout.put(ch);
    cout.flush();
    getch();
}

```

Example : io2.cpp

```

#include<iostream.h>
#include<conio.h>
#include<string.h>
void main()
{
    char st[20];
    clrscr();
    cout<<"Enter any String : ";
    cin.get(st,20);
    cout<<"Given String : ";
    cout.write(st,strlen(st));
    cout.flush();
    getch();
}

```

Formatted I/O operations :

C++ supports a number of features that could be used for formatting the output, these features include .

- 1 . i/o manipulators
- 2 . ios flags, functions

io manipulators :

setw : It changes the field width to n .

Syntax : setw(int n);

setfill : It changes fill character to ch.

Syntax : setfill(char ch);

setprecision : It changes the precision to n places after the decimal point for floating values.

Syntax : setprecision(int n);

setbase :

It changes the base to n (base value 8 for octal, 10 for decimal, 16 for hexa decimal)

Syntax : setbase(int n);

Example : *io3.cpp*

```
#include<iostream.h>
#include<conio.h>
#include<iomanip.h>
```

```

void main()
{
    int n=100;
    cout<<setw(8)<<n<<endl;
    cout<<setfill('*');
    cout<<setw(8)<<n<<endl;
    float f=18.14;
    cout<<f<<endl;
    cout<<setprecision(2);
    cout<<f<<endl;
}

```

Example : io4.cpp

```

#include<iostream.h>
#include<conio.h>
#include<iomanip.h>
void main()
{
    int m=100;
    int n=200;
    clrscr();
    cout<<"m = "<<m<<endl;
    cout<<"n = "<<n<<endl;
    cout<<setbase(8);
    cout<<"Octal value of m = "<<m<<endl;
    cout<<"Octal value of n = "<<n<<endl;
    cout<<setbase(10);
    cout<<"Decimal value of m = "<<m<<endl;
}

```

```

cout<<"Decimal value of n = "<<n<<endl;
cout<<setbase(16);
cout<<"Hexa decimal value of m = "<<m<<endl;
cout<<"Hexa decimal value of n = "<<n<<endl;
}

```

setiosflags : It sets the specified ios flag.

Syntax : setiosflags(flag);

resetiosflags : It removes the specified ios flag.

Syntax : resetiosflags(flag);

Flag	Description

ios :: left	Left align
ios :: right	Right align
ios :: internal	It separates base indicator
ios :: showpoint	it displays decimal point for floating values
ios :: scientific	Scientific notation

ios :: fixed	Fixed floating point notation
ios :: dec	It changes base to decimal
ios :: oct	It changes base to octal
ios :: hex	It changes base to hexadecimal
ios :: showbase	It displays base indicator
ios :: showpos	It displays plus sign before the positive value
ios :: uppercase and	It displays upper case letters for hexadecimal values scientific values.

Example : io5.cpp

```
#include<iostream.h>
#include<conio.h>
#include<iomanip.h>
```

```

void main()
{
    int n=123;
    clrscr();
    cout<<n<<endl;
    cout<<setfill('.');
    cout<<setiosflags(ios::left);
    cout<<setw(8)<<n<<endl;
    cout<<setiosflags(ios::right);
    cout<<setw(8)<<n<<endl;
    cout<<setiosflags(ios::showpos);
    cout<<setiosflags(ios::internal);
    cout<<setw(8)<<n<<endl;
    cout<<resetiosflags(ios::showpos);
    float f=123.345;
    cout<<f<<endl;
    cout<<setprecision(2);
    cout<<setiosflags(ios::showpoint);
    cout<<f<<endl;
    cout<<setiosflags(ios::scientific);
    cout<<f<<endl;
    cout<<setiosflags(ios::uppercase);
    cout<<f<<endl;
    cout<<setiosflags(ios::fixed);
    cout<<f<<endl;
    getch();
}

```

Example : io6.cpp

```
#include<iostream.h>
#include<conio.h>
#include<iomanip.h>
void main()
{
    int n=100;
    clrscr();
    cout<<"n : "<<n<<endl;
    cout<<setiosflags(ios::showbase);
    cout<<setiosflags(ios::oct);
    cout<<"Octal value : "<<n<<endl;
    cout<<setiosflags(ios::dec);
    cout<<"Decimal value : "<<n<<endl;
    cout<<setiosflags(ios::hex);
    cout<<"Hexa Decimal value : "<<n<<endl;
    getch();
}
```

ios class functions :

width : It changes the field width to n.

Syntax : width(int n);

fill : It changes fill character to ch.

Syntax : fill(char ch);

precision : It changes the precision to n places after the decimal point for floating values.

Syntax : precision(int n);

setf : It sets specified ios flag

Syntax : setf(flag);

unsetf : It removes the specified ios flag

Syntax : unsetf(flag);

Example : io7.cpp

```
#include<iostream.h>
#include<conio.h>
#include<iomanip.h>
void main()
```

```
{
    int n=123;
    clrscr();
    cout<<n<<endl;
    cout.fill('.');
    cout.setf(ios::left);
    cout.width(8);
    cout<<n<<endl;
    cout.setf(ios::right);
    cout.width(8);
    cout<<n<<endl;
    cout.setf(ios::showpos);
    cout.setf(ios::internal);
    cout.width(8);
    cout<<n<<endl;
    cout.unsetf(ios::showpos);
    float f=123.345;
    cout<<f<<endl;
    cout.precision(2);
    cout.setf(ios::showpoint);
    cout<<f<<endl;
    cout.setf(ios::scientific);
    cout<<f<<endl;
    cout.setf(ios::uppercase);
    cout<<f<<endl;
    cout.setf(ios::fixed);
    cout<<f<<endl;
    getch();
}
```


Files :

A file is a collection of records (or) related data stored in a particular area on the disk.

A program typically involves the following kinds of data communication.

- 1 Data transfer between console unit and program
- 2 Data transfer between program and a disk file.

We have already discussed the technique of handling data communication between console unit and program.

The IO system of C++ handles file operations, that are very much similar to the console IO operations. It uses file streams as an interface between programs and files. The stream that supplies data to the program is called as input filestream and the stream that receives data from the program is called as output filestream.

Classes for file stream operations :

The IO system of C++ contains a Hierarchy of classes that define the file handling methods. These include ifstream, ofstream and fstream. These classes are derived from fstreambase and the corresponding iostream classes.

filebuf : It's purpose is to set the buffers to read and write data.

fstream base : It provides operations common to file streams. It provides as a base for ifstream, ofstream classes.

ifstream : It provides input file operations.

ofstream : It provides output file operations.

fstream : It provides simultaneous input and output file operations.

Opening files using constructors :

We know that a constructor is used to initialize an object while it is being created. Here a file name is used to initialize the file stream object. It involves the following 2 steps.

- 1 Create a file stream object to manage the stream using the appropriate class that is to say that the class ofstream is used to create the output file stream and the class ifstream is used to create input file stream.
- 2 Initialize the file stream object with the desire file stream.

Example : file1.cpp

To create a new file or to open an existing file

```
#include<fstream.h>
#include<conio.h>
void main()
{
    clrscr();
    ofstream o("A.txt");
    if(o)
        cout<<"File is opened";
    else
        cout<<"Unable to open a file";
    getch();
}
```

Example : file2.cpp

To store data into a file

```
#include<fstream.h>
```

```

#include<conio.h>
void main()
{
    clrscr();
    ofstream o("A.txt");
    o<<"Welcome to CPP Files";
    cout<<"Data stored (to see the data open A.txt)";
    getch();
}

```

Example : file3.cpp

To read data from the existing file

```

#include<fstream.h>
#include<conio.h>
void main()
{
    char st[20];
    clrscr();
    ifstream i("A.txt");
    // i>>st; this statement cannot read white spaces
    i.getline(st,20); // this statement can read white spaces
    cout<<st<<endl;
    cout<<"Data read successfully";
    getch();
}

```

Example : file4.cpp

To read data from the keyboard and to write data into a file

```

#include<fstream.h>
#include<conio.h>
void main()
{
    char ch;
    clrscr();
    ofstream o("A.txt");
    cout<<"Enter data into the file (ctrl+Z to stop) : ";
}

```

```

ch=cin.get();
while(ch!=EOF)
{
    o<<ch;
    ch=cin.get();
}
cout<<"Data stored successfully";
}

```

Example : file5.cpp

To read data from the existing file and display it on monitor

```

#include<fstream.h>
#include<conio.h>
#include<dos.h>
void main()
{
    char ch;
    clrscr();
    ifstream i("A.txt");
    cout<<"Data reading from file"<<endl;
    ch=i.get();
    while(ch!=EOF)
    {
        cout<<ch;
        ch=i.get();
        delay(500);
    }
    cout<<"Data red successfully";
}

```

EOF :

It is a constant, indicating that end-of-file has been reached on a file
To get this character from keyboard press F6 or Ctrl + Z.

open: It is a fstream function and is used to open a file.

Syntax : streamobject.open("file_name");

close: It is a fstream function which closes the specified file .

Syntax : streamobject.close();

Example : file3.cpp

To write data into the file and to read from that file and display it on monitor

```
#include<fstream.h>
#include<conio.h>
#include<dos.h>
void main()
{
    char ch;
    clrscr();
    ofstream o;
    ifstream i;
    o.open("A.txt");
    cout<<"Enter data (ctrl+Z to stop) : ";
    ch=cin.get();
    while(ch!=EOF)
    {
        o<<ch;
        ch=cin.get();
    }
    o.close();
    cout<<endl<<"Data stored"<<endl;
    cout<<endl<<endl<<"Data reading from file"<<endl;
    i.open("A.txt");
    ch=i.get();
    while(ch!=EOF)
    {
        cout<<ch;
```

```

        ch=i.get();
        delay(100);
    }
    i.close();
    cout<<endl<<"Data red successfully";
    getch();
}

```

Example : file7.cpp

```

#include<fstream.h>
#include<conio.h>
#include<process.h>
#include<dos.h>
void main()
{
    char fn[20],ch;
    clrscr();
    cout<<"Enter File name : ";
    cin>>fn;
    ifstream ifs(fn);
    if(!ifs)
    {
        cout<<"File does not exists";
        getch();
        exit(0);
    }
    cout<<endl<<"Data reading from file"<<endl<<endl;
    ch=ifs.get();
    while(ch!=EOF)
    {
        cout<<ch;
        ch=ifs.get();
        delay(100);
    }
    cout<<endl<<"Data red successfully";
    getch();
}

```

File modes :

We have used ifstream and ofstream constructors and the function open to create new files as well as to open the existing files. Remember in both these methods we can use only one argument that was the file name. If we want to open a file using fstream object, the open function can take 2 arguments.

1. File name
2. File mode.

Syntax :

```
fstreamobject.open( "file_name" , file_mode);
```

File mode	Description
ios :: app	Appends to the end of file
ios :: ate	Goto end of file on opening
ios :: binary	Binary file

ios :: in	Open file for reading only
ios :: nocreate	Open fails if the file doesn't exists
ios :: noreplace	Open fails if the file already exists
ios :: out	Open file for writing only
ios :: trunk	It deletes the contents of file if it is exists

- 1 Opening a file in ios :: out mode also, opens in the ios :: trunk mode by default.
- 2 Both ios :: app and ios :: ate takes us to the end of file when it is opened. The difference between these 2 parameters is that ios :: app allows us to add data to the end of file only. Where as, ios :: ate permits us to add data or modify the data any where in the file. In both the case a file is created by the specified name if it does not exists.
- 3 The fstream classes doesnt provide a mode by default, therefore we must provide the mode explicitly when using the object of fstream classes.
- 4 The mode can combine 2 or more parameters using bitwise operator OR (|).

Example : file8.cpp

```
#include<fstream.h>
#include<conio.h>
#include<dos.h>
void main()
{
    char ch;
    clrscr();
```

```

fstream fs;
fs.open("A.txt",ios::app);
cout<<"Enter data(Ctrl+Z to stop) : ";
ch=cin.get();
while(ch!=EOF)
{
    fs<<ch;
    ch=cin.get();
}
fs.close();
cout<<endl<<"Data stored Successfully"<<endl;
cout<<endl<<"Reading data from File"<<endl;
fs.open("A.txt",ios::in);
ch=fs.get();
while(ch!=EOF)
{
    cout<<ch;
    ch=fs.get();
    delay(50);
}
fs.close();
cout<<endl<<"Data read successfully";
getch();
}

```

tellg() : It gives the current file pointer position in the input mode.

Syntax : streamobject.tellg();

tellp() : It gives the current file pointer position in the output mode.

Syntax : streamobject.tellp()

seekg: It moves the file pointer to a specified location in file input

mode

Syntax : streamobject.seekg(long offset,reference_position);

seekp : It moves the file pointer to a specified location in file output mode

Syntax : streamobject.seekp(long offset,reference_position);

Reference position	Description
ios :: beg	Beginning of the file
ios :: cur	Current position of the file
ios :: end	End of file

Example : file9.cpp

```
#include<fstream.h>
#include<conio.h>
#include<dos.h>
void main()
{
    char ch;
    clrscr();
    fstream fs;
    fs.open("A.txt",ios::app|ios::in);
    cout<<"Enter data (Ctrl+Z to stop) : ";
    ch=cin.get();
    while(ch!=EOF)
    {
        fs<<ch;
        ch=cin.get();
    }
    cout<<endl<<"Current file pointer position : "<<fs.tellp()<<endl;
    // fs.seekg(0,ios::beg);
```

```

// fs.seekg(10,ios::beg);
fs.seekg(-10,ios::cur);
cout<<endl<<endl<<"Data Stored Succcessfully"<<endl;
cout<<endl<<"Reading data from the file"<<endl;
cout<<endl<<"Current file pointer position : "<<fs.tellp();
ch=fs.get();
while(ch!=EOF)
{
    cout<<ch;
    ch=fs.get();
    delay(50);
}
cout<<endl<<"Current file pointer position : "<<fs.tellp();
fs.close();
cout<<endl<<endl<<"Data red successfully";
getch();
}

```

Reading and Writing class objects :

write:

It is a file output stream function and used to write objects (data blocks) to an output file.

Syntax :

```
streamobject.write((char *)&obj,sizeof(obj));
```

read :

It is a file input stream function and used to read objects from input file.

Syntax :

```
streamobject.read((char *)&obj,sizeof(obj));
```

Example : file10.cpp

```
#include<fstream.h>
#include<iostream.h>
#include<conio.h>
#include<iomanip.h>
class emp
{
    private:
        int eno;
        char ename[20];
        float sal;
    public:
        void accept();
        void disp();
};
void emp::accept()
{
    cout<<"Enter employ number : ";
    cin>>eno;
    cout<<"Enter employ name    : ";
    cin.ignore();
    cin.getline(ename,20);
    cout<<"Enter employ salary : ";
    cin>>sal;
}
void emp::disp()
```

```

{
    cout<<"Employ number : "<<eno<<endl;
    cout<<"Employ name   : "<<ename<<endl;
    cout<<"Employ salary : "<<sal<<endl;
}
void main()
{
    emp e;
    fstream fs1,fs2;
    cout<<setprecision(2);
    cout<<setiosflags(ios::showpoint);
    cout<<setiosflags(ios::fixed);
    clrscr();
    fs1.open("emp.dat",ios::out);
    cout<<"Enter Employee Details"<<endl<<endl;
    e.accept();
    fs1.write((char *)&e,sizeof(e));
    cout<<endl<<"Data stored Successfully...."<<endl;
    fs1.close();
    cout<<endl<<"Reading      data      from      the
file"<<endl<<endl;
    cout<<"Employee Details"<<endl;
    cout<<"-----"<<endl;
    fs2.open("emp.dat",ios::in);
    e.disp();
    getch();
}

```

eof :

It returns a non zero if end of file.

Syntax : int eof();

Example : **file11.cpp**